

DYNOPIPE: Heterogeneous Edge-Cloud LLM Serving with Dynamically Orchestrated Pipeline Boundaries

Yanying Lin^{1,2,3}, Baicheng Chen³, Xinyu Zhang³, Chengzhong Xu⁴, and Kejiang Ye^{1,*}

¹Shenzhen Institutes of Advanced Technology, Chinese Academy of Sciences

²University of Chinese Academy of Sciences ³University of California San Diego ⁴University of Macau

Abstract—Large language model (LLM) deployment at the network edge faces a fundamental paradox: applications require full-scale models for sophisticated reasoning, yet edge devices impose severe resource constraints across computation, memory, and network. Existing approaches fail to effectively orchestrate resources across the edge-cloud continuum, leaving capacity underutilized while struggling with heterogeneous and volatile distributed environments. We present DYNOPIPE, an adaptive edge-cloud system that addresses these constraints through dynamic pipeline parallelism with shifting computational boundaries. DYNOPIPE tackles three core challenges: structural heterogeneity causing 94% pipeline idle time, temporal resource volatility invalidating static partitioning, and boundary migration overhead trapping systems in suboptimal configurations. Through boundary-constrained pipeline construction, proactive multi-configuration orchestration, and hierarchical state management, DYNOPIPE eliminates the memory wall while preserving data locality, achieving 10.1× throughput improvement over edge-only baselines and 1.6× over cloud-only execution, with 99.2% latency reduction.

1. Introduction

Edge-centric LLM deployment faces a fundamental trade-off: applications need full-scale models for complex reasoning, but edge devices lack the resources to run them. This constraint manifests across the edge-cloud continuum, where limited local compute and memory resources conflict with the substantial requirements of modern LLMs, while constrained uplink bandwidth creates additional bottlenecks for cloud offloading [1–3]. Furthermore, privacy-critical applications in domains such as healthcare and finance necessitate local processing, making straightforward cloud migration strategies impractical [4–6].

Existing solutions exhibit fundamental architectural limitations that preclude effective edge LLM deployment. Cloud offloading creates prohibitive network latency for interactive applications [1, 7, 8] while exposing privacy sensitive raw data to servers over public network [4, 9, 10]. Model compression through quantization and distillation [11–14] incurs severe accuracy degradation for complex reasoning tasks requiring full representational capacity. Edge federation encounters quadratic synchronization overhead as the number of devices scales [15–17], causing memory fragmentation and communication bottlenecks that eliminate resource pooling benefits. These inherent trade-offs necessitate a paradigm that overcomes single-domain constraints via cross-boundary resource orchestration.

Kejiang Ye is the corresponding author.

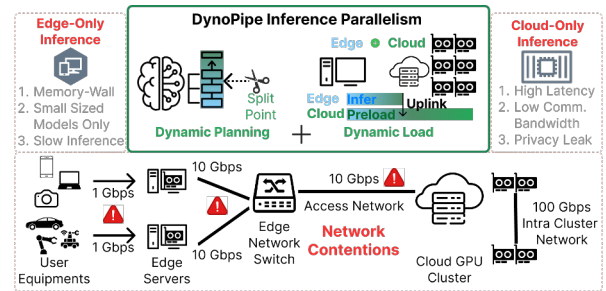


Fig. 1: DYNOPIPE addresses the fundamental trilemma of edge LLM deployment: computational bottlenecks at the edge, network latency in cloud offloading, and resource fragmentation across the edge-cloud continuum through adaptive pipeline parallelism with dynamic boundary shifting.

DYNOPIPE Proposal. As illustrated in Fig. 1, we propose DYNOPIPE, a system that dynamically orchestrates LLM inference across edge-cloud boundaries through adaptive pipeline parallelism with shifting computation boundaries. DYNOPIPE strategically partitions inference stages based on real-time resource availability: the edge executes privacy-sensitive embeddings while transmitting only compact activation tensors to the cloud for computation-intensive operations. This boundary-shifting design circumvents the memory wall by offloading heavyweight computations while preserving data locality.

Challenges. First, *structural heterogeneity* creates fundamental pipeline imbalances where edge-cloud computational asymmetry prevents effective workload distribution [18–20]. Edge devices with limited memory bandwidth cannot match cloud servers’ computational throughput, creating severe bottlenecks at boundary stages. The memory wall effect [21] forces edge components to operate at lower utilization rates while cloud resources remain underutilized due to pipeline synchronization constraints, resulting in 94% pipeline idle time (§2.1). This asymmetric architecture forces the entire pipeline to operate at the slowest component’s pace, fundamentally limiting throughput despite abundant cloud resources.

Second, *temporal resource volatility* invalidates static boundary placement through dynamic interference patterns that create cascading performance degradation [7, 22]. Thermal throttling creates computational performance degradation of up to 25%, while network-compute correlation creates feedback loops where

DynoPipe: 异构边缘云 LLM 提供动态编排的管道边界

林彦英^{1,2,3}、陈百成³、张新宇³、徐成中⁴和叶克江^{1,*} ¹中国科学院深圳先进技术研究院
²中国科学院大学³加州大学圣地亚哥分校⁴澳门大学

Abstract— 网络边缘的大语言模型 (LLM) 部署面临着一个基本悖论：应用程序需要完整的模型来进行复杂的推理，但边缘设备在计算、内存和网络方面施加了严格的资源限制。现有方法无法有效地跨边缘云连续体协调资源，导致容量利用率不足，同时还要应对异构且不稳定的分布式环境。我们提出了 DynoPipe，这是一种自适应边缘云系统，它通过动态管道并行性和移动计算边界来解决这些限制。DynoPipe 解决了三个核心挑战：导致 94% 管道空闲时间的结构异构性、导致静态分区失效的时间资源波动以及次优配置中的边界迁移开销捕获系统。通过边界约束管道构建、主动多配置编排和分层状态管理，DynoPipe 消除了内存墙，同时保留了数据局部性，与仅边缘基线相比实现了 10.1× 吞吐量改进，比仅云执行提高了 1.6×，延迟减少了 99.2%。

一、简介

以边缘为中心的 LLM 部署面临着一个基本的权衡：应用程序需要完整的模型来进行复杂的推理，但边缘设备缺乏运行它们的资源。这种限制在边缘-云连续体中表现出来，其中有限的本地计算和内存资源与现代大语言模型的实质性要求相冲突，而受限的上行链路带宽为云卸载造成了额外的瓶颈[1-3]。此外，医疗保健和金融等领域的隐私关键应用程序需要本地处理，使得简单的云迁移策略不切实际[4-6]。

现有解决方案表现出基本的架构限制，阻碍了有效的边缘 LLM 部署。云卸载会给交互式应用程序带来令人望而却步的网络延迟 [1,7,8]，同时通过公共网络将隐私敏感的原数据暴露给服务器 [4,9,10]。对于需要完整表示能力的复杂推理任务，通过量化和蒸馏进行模型压缩[11-14]会导致严重的准确性下降。随着设备数量的扩展，边缘联合会遇到二次同步开销[15-17]，导致内存碎片和通信瓶颈，从而消除了资源池的优势。这些固有的权衡需要一种通过跨界资源编排克服单域限制的范例。

Kejiang Ye is the corresponding author.

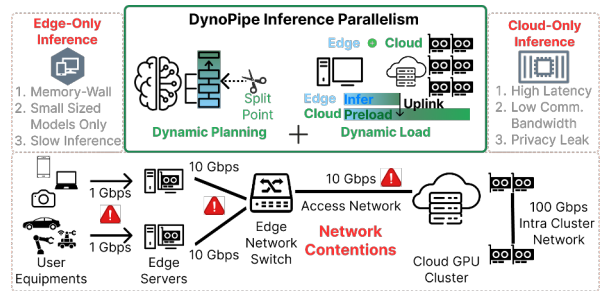


图 1: DynoPipe 通过具有动态边界转移的自适应管道并行性解决了边缘 LLM 部署的基本三难困境：边缘的计算瓶颈、云卸载中的网络延迟以及跨边缘-云连续体的资源碎片。

DynoPipe 提案。如图 1 所示，我们提出了 DynoPipe，这是一个通过自适应管道并行性和移动计算边界动态协调跨边缘云边界的 LLM 推理的系统。DynoPipe 根据实时资源可用性对推理阶段进行策略性划分：边缘执行隐私敏感的嵌入，同时仅将紧凑的激活张量传输到云以进行计算密集型操作。这种边界转移设计通过卸载重量级计算同时保留数据局部性来绕过内存墙。

挑战。首先，*structural heterogeneity*造成了基本的管道不平衡，其中边缘云计算不对称阻碍了有效的工作负载分配[18-20]。内存带宽有限的边缘设备无法匹配云服务器的计算吞吐量，从而在边界阶段造成严重的瓶颈。内存墙效应 [21] 迫使边缘组件以较低的利用率运行，而云资源由于管道同步限制而仍未得到充分利用，导致 94% 的管道空闲时间（第 2.1 节）。这种不对称架构迫使整个管道以最慢的组件速度运行，尽管云资源丰富，但从根本上限制了吞吐量。

其次，*temporal resource volatility* 通过动态干扰模式使静态边界放置失效，从而导致级联性能下降 [7, 22]。热节流导致计算性能下降高达 25%，而网络计算相关性会创建反馈循环，其中

queuing delays trigger retransmissions, amplifying contention and causing oscillatory behavior. These temporal variations shift optimal boundary placement by multiple stages within minutes, making static optimization ineffective for online workloads (§2.2). The fundamental issue is that edge-cloud systems exhibit non-linear performance characteristics where small resource fluctuations can trigger threshold effects, causing dramatic shifts in optimal partitioning strategies that static approaches cannot anticipate or adapt to.

Third, *boundary migration overhead* traps the system in suboptimal configurations by creating prohibitive state transfer costs that exceed the benefits of adaptation [23–25]. Stateful reconfiguration incurs multi-dimensional overhead where transferring accumulated state requires tens of seconds to rebuild the cache, creating service interruptions exceeding acceptable latency bounds (§2.3). The migration cost prevents dynamic adaptation to changing resource conditions, undermining the elasticity required for efficient edge–cloud collaboration. This creates a fundamental paradox: while dynamic adaptation is necessary for optimal performance, the cost of adaptation itself becomes the primary performance bottleneck, forcing systems to remain in increasingly suboptimal configurations as workload conditions evolve.

Methodology. DYNPIPE addresses these challenges through a boundary-aware optimization framework operating across three interconnected dimensions. *Boundary-constrained pipeline construction* (§4.1) formulates pipeline partitioning as an optimization problem that models heterogeneous transition costs across edge-cloud boundaries, accounting for computational asymmetry, memory hierarchy mismatch, and network topology disparities. The algorithm generates boundary configuration portfolios (bandwidth-constrained, compute-constrained, memory-constrained) using dynamic programming for precise optimization under resource asymmetry.

Proactive multi-configuration orchestration (§4.2) addresses temporal resource volatility through pre-computed boundary portfolios that monitor resource conditions via lightweight telemetry and switch configurations when thresholds are violated. Under bandwidth contention, it selects activation-minimal boundaries; under abundant bandwidth, it shifts boundaries earlier to maximize cloud utilization, avoiding prohibitive state reconstruction overhead.

Hierarchical state orchestration (§4.3) enables rapid boundary migration through parameter overlap caching and bandwidth-aware state partitioning. The system maintains overlapping parameter sets across domains for sub-millisecond GPU-to-GPU transfers via PCIe/NVLink, while exploiting edge-cloud link asymmetry by streaming state transfers asynchronously through surplus downlink capacity. Temporal state compression through differential KV cache representations reduces reconfiguration overhead from seconds to milliseconds while maintaining cache continuity across boundary transitions.

We evaluated DYNPIPE on real-world workloads using a 16-

server edge-cloud testbed. Results demonstrate that DYNPIPE achieves efficient full-scale LLM inference by overcoming memory wall constraints through edge-cloud collaboration. DYNPIPE reduces TTFT latency by 98.9% (from 68.53s to 0.74s) and improves throughput by 10.1× compared to edge-only solutions. Compared to cloud-only execution, DYNPIPE achieves 1.6× throughput improvement by converting single-GPU serialization into edge-cloud pipeline parallelism, significantly reducing queuing overhead as the dominant latency component. By eliminating the memory wall through collaborative pipeline architecture, DYNPIPE reduces request waiting latency by 99.2%.

Contributions. This paper makes the following contributions:

- A boundary-aware pipeline parallelism system that enables efficient LLM inference across heterogeneous edge-cloud environments through dynamic computation orchestration and adaptive resource allocation under structural asymmetry constraints.
- A profiling-driven heuristic orchestration framework that addresses computational asymmetry, memory hierarchy mismatch, and network topology disparities through boundary-constrained partitioning and proactive portfolio-based configuration selection, with empirically validated robustness under diverse edge-cloud operating regimes.
- A hierarchical state orchestration mechanism that enables rapid boundary migration through parameter overlap caching and bandwidth-aware state partitioning, reducing reconfiguration overhead from seconds to milliseconds while maintaining inference continuity across dynamic bottleneck transitions.

2. Background and Motivation

Large language models face fundamental deployment challenges across the edge-cloud continuum. Edge deployments suffer from memory wall constraints [21], while cloud-only solutions suffer from prohibitive network latency [1, 7, 8] and privacy risks [4, 5]. Pipeline parallelism emerges as a natural solution—partitioning computation so that privacy-sensitive operations remain local while compute-intensive tasks leverage cloud resources—but applying it across heterogeneous edge-cloud boundaries introduces three critical challenges.

2.1 Resource Competition for Online Workloads

Pipeline parallelism partitions computation across heterogeneous nodes, enabling edge devices to handle privacy-sensitive operations while cloud resources execute compute-intensive tasks [18, 26, 27]. However, edge deployments face fundamental constraints: limited GPU memory restricts model capacity, concurrent request handling intensifies memory pressure and bandwidth contention, and these resource boundaries fluctuate continuously, invalidating static partitioning assumptions [28, 29].

Multi-model deployment constraints exacerbate resource competition. Edge devices often require simultaneous support for

排队延迟会触发重传，加剧争用并导致振荡行为。这些时间变化会在几分钟内将最佳边界放置改变多个阶段，使得静态优化对于在线工作负载无效 (§2.2)。根本问题是边缘云系统表现出非线性性能特征，其中小的资源波动可能会触发阈值效应，导致最佳分区策略发生巨大变化，而静态方法无法预测或适应。

第三，*boundary migration overhead* 通过创建超出适应收益的令人望而却步的状态转移成本，使系统陷入次优配置[23-25]。有状态重新配置会产生多维开销，其中传输累积状态需要数十秒来重建缓存，从而造成超出可接受延迟范围的服务中断（第2.3节）。迁移成本阻碍了对不断变化的资源条件的动态适应，从而破坏了高效边缘云协作所需的弹性。这就产生了一个基本的悖论：虽然动态适应对于最佳性能是必要的，但适应本身的成本却成为主要的性能瓶颈，随着工作负载条件的变化，迫使系统保持在越来越次优的配置中。

方法论。DynoPipe 通过跨三个互连维度运行的边界感知优化框架来应对这些挑战。*Boundary-constrained pipeline construction* (§4.1) 将管道分区表述为一个优化问题，对跨边云边界的异构转换成本进行建模，并考虑计算不对称、内存层次结构不匹配和网络拓扑差异。该算法使用动态规划生成边界配置组合（带宽受限、计算受限、内存受限），以在资源不对称下进行精确优化。

Proactive multi-configuration orchestration (§4.2) 通过预先计算的边界组合解决时间资源波动性，这些组合通过轻量级遥测监控资源状况，并在违反阈值时切换配置。在带宽争用的情况下，它选择激活最小边界；在充足的带宽下，它会更早地转移边界以最大化云利用率，避免过高的状态重建开销。

Hierarchical state orchestration (§4.3) 通过参数重叠缓存和带宽感知状态分区实现快速边界迁移。该系统通过 PCIe/NVLink 维持跨域的重叠参数集，以实现亚毫秒级 GPU 到 GPU 传输，同时通过剩余下行链路容量异步流式传输状态传输，从而利用边缘云链路不对称性。通过差异 KV 缓存表示的时间状态压缩将重新配置开销从几秒减少到几毫秒，同时保持跨边界转换的缓存连续性。

我们使用 16-

服务器边缘云测试平台。结果表明，DynoPipe 通过边云协作克服内存墙限制，实现了高效的全面 LLM 推理。与仅边缘解决方案相比，DynoPipe 将 TTFT 延迟减少了 98.9%（从 68.53 秒减少到 0.74 秒），并将吞吐量提高了 10.1x。与纯云执行相比，DynoPipe 通过将单 GPU 串行化转换为边缘云管道并行性，实现了 1.6x 吞吐量提升，显著减少了作为主要延迟组件的排队开销。通过协作管道架构消除内存墙，DynoPipe 将请求等待延迟减少了 99.2%。

贡献。本文做出以下贡献：

- 边界感知管道并行系统，通过结构不对称约束下的动态计算编排和自适应资源分配，实现跨异构边缘云环境的高效 LLM 推理。
- 一种分析驱动的启发式编排框架，通过边界约束分区和主动的基于组合的配置选择来解决计算不对称、内存层次结构不匹配和网络拓扑差异，并在不同的边缘云操作机制下具有经过经验验证的鲁棒性。
- 分层状态编排机制，通过参数重叠缓存和带宽感知状态分区实现快速边界迁移，将重新配置开销从几秒减少到几毫秒，同时保持动态瓶颈转换的推理连续性。

2. 背景和动机

大语言模型面临着跨边缘云连续体的基本部署挑战。边缘部署受到内存墙限制 [21]，而纯云解决方案则面临令人望而却步的网络延迟 [1, 7, 8] 和隐私风险 [4, 5]。管道并行性作为一种自然的解决方案出现——对计算进行分区，以便隐私敏感的操作保留在本地，而计算密集型任务则利用云资源——但跨异构边缘云边界应用它会带来三个关键挑战。

2.1 在线工作负载的资源竞争

管道并行性跨异构节点划分计算，使边缘设备能够处理隐私敏感的操作，而云资源执行计算密集型任务[18,26,27]。然而，边缘部署面临着基本的限制：有限的GPU内存限制了模型容量，并发请求处理加剧了内存压力和带宽争用，并且这些资源边界不断波动，使得静态分区假设失效[28, 29]。

Multi-model deployment constraints 加剧资源竞争。边缘设备通常需要同时支持

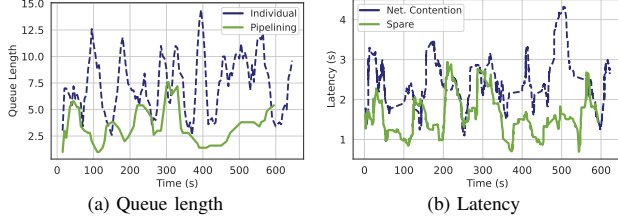


Fig. 2: The latency and queue length of the LLaMA2-7B inference process. (a) Queue length of model inference individually vs. pipeline parallelism. (b) Latency of model inference in network contention and spare bandwidth.

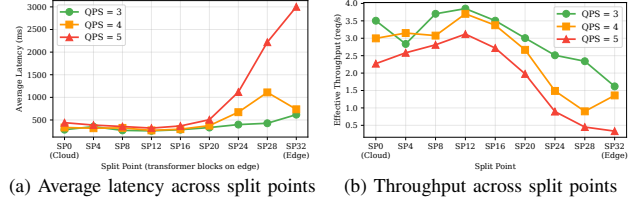
diverse tasks (speech recognition, image processing, NLP), but resource limitations make concurrent high-performance model deployment impractical, and frequent model loading/unloading further increases latency.

The effectiveness of caching is reduced due to the continuous growth of KV caches in LLMs. Models with chain-of-thought (CoT) phases generate thousands of tokens as context before producing output, demanding substantial memory for intermediate storage. Edge devices with limited capacity frequently trigger cache eviction, causing pipeline stalls for KV reconstruction. Pipeline parallelism addresses this by partitioning models into stages and enabling concurrent batch processing, reducing queue length by approximately 79.8% (Fig. 2a).

Bandwidth contention is exacerbated by concurrent multi-task inference. Edge LLM services typically share the same router, so concurrent tasks compete for bandwidth: with four concurrent inference tasks, effective per-task bandwidth drops from 10 Gbps to below 2.5 Gbps, increasing activation transmission latency by 4.9 $\times$ (from 1.68 ms to 8.2 ms). This invalidates the communication latency assumptions underlying static pipeline design. Our case study shows that alleviating bandwidth contention reduces end-to-end latency by 43% (Fig. 2b).

Cloud-only deployment limitations emerge under realistic load conditions. While cloud-only execution (all layers on cloud GPU) achieves optimal single-request latency (179ms for LLaMA2-7B), it degrades catastrophically under concurrent load: at QPS=5, queueing accounts for 62% of total latency (295ms out of 478ms) due to single-GPU serialization. In contrast, edge-cloud pipeline collaboration (split at layer 12) reduces queueing to 26% of total latency through pipeline parallelism, achieving 39% lower end-to-end latency and 64% higher throughput despite introducing network transfer overhead (Fig. 3). This demonstrates that the fundamental advantage of edge-cloud collaboration lies not in resource augmentation but in the throughput multiplier effect of pipeline parallelism under load.

Insight 1. *Dynamic resource contention at the edge, due to concurrent multi-model deployment and bandwidth competition, invalidates static partitioning assumptions. This necessitates adaptive edge-cloud orchestration that dynamically responds to real-time resource availability and workload characteristics.*



(a) Average latency across split points (b) Throughput across split points

Fig. 3: Split-point (SP) study on LLaMA2-7B under Poisson arrivals (QPS=3/4/5). Cloud-only (SP=0) and edge-only (SP=32) are consistently suboptimal; intermediate split points achieve the best latency-throughput trade-off through pipeline parallelism.

2.2 Heterogeneous Architecture Constraints

Edge-cloud architectures exhibit fundamental resource asymmetries that create structural constraints on pipeline design. These three-dimensional disparities—computation (0.48 $\times$), memory (2.5 $\times$), and communication (80 $\times$)—fundamentally constrain pipeline adaptability through cascading effects that compound across system layers.

Computation imbalance creates stage latency skew that prevents efficient pipeline balancing. H100 GPUs (989 TFLOPS) versus NVIDIA Thor at the edge (2070 TOPS) reflect a 0.48 $\times$ compute disparity. During LLaMA2-70B prefilling, this disparity manifests as asymmetric execution patterns: attention layers exhibit 2.7 $\times$ latency variance between domains, while feed-forward networks show 1.8 $\times$ variance. The critical insight is that different operator types experience varying degrees of heterogeneity amplification—attention operations suffer disproportionately on edge devices due to memory bandwidth limitations, while embedding layers show minimal performance degradation. This operator-specific heterogeneity creates irregular pipeline bubbles that resist traditional load balancing, producing up to 67% theoretical idle time as fast stages wait for heterogeneity-amplified slow stages.

Memory hierarchy mismatch forces architectural rigidity through constrained stage granularity. Edge DRAM (64GB, 51.2 GB/s) versus cloud HBM (140GB, 3.35 TB/s) creates a 65 $\times$ bandwidth disparity that compounds memory capacity constraints. This mismatch manifests in two critical ways: (1) *Stage fragmentation*—edge memory constraints force fine-grained partitioning that reduces computation-communication overlap opportunities, and (2) *Cache hierarchy violations*—KV cache access patterns optimized for HBM exhibit severe performance degradation on DRAM, creating 4.2 $\times$ latency penalties for long-context inference. The hierarchy mismatch prevents flexible stage reassignment: stages optimized for HBM access patterns cannot efficiently migrate to DRAM-based edge nodes without fundamental algorithmic restructuring.

Communication topology asymmetry creates bandwidth-latency trade-offs that dominate runtime characteristics. Edge-cloud links (10 Gbps, 15ms RTT) versus cloud interconnects (800 Gbps RDMA, 1.5 μ s RTT) exhibit 80 $\times$ bandwidth and 10,000 $\times$ latency disparities. This asymmetry makes tensor parallelism prohibitively expensive across edge-cloud bound-

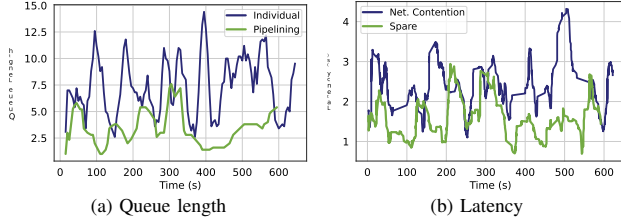


图 2: LLaMA2-7B 推理过程的延迟和队列长度。(a) 单独模型推理的队列长度与管道并行性。(b) 网络争用和空闲带宽中模型推理的延迟。

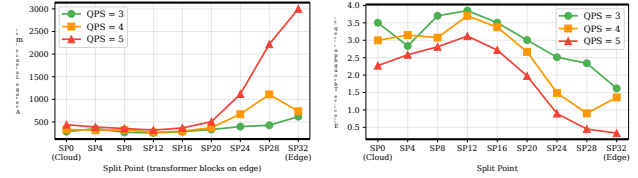
多样化的任务（语音识别、图像处理、NLP），但资源限制使得并发高性能模型部署不切实际，频繁的模式加载/卸载进一步增加了延迟。

The effectiveness of caching is reduced 由于 LLM 中 KV 缓存的不断增长。具有思想链（CoT）阶段的模型在产生输出之前会生成数千个令牌作为上下文，需要大量内存用于中间存储。容量有限的边缘设备经常触发缓存驱逐，导致 KV 重建的管道停顿。管道并行性通过将模型划分为多个阶段并启用并发批处理来解决这个问题，从而将队列长度减少约 79.8%（图 2a）。

Bandwidth contention 并发多任务推理会加剧这种情况。Edge LLM 服务通常共享同一路由器，因此并发任务会争夺带宽：在四个并发推理任务的情况下，每个任务的有效带宽从 10 Gbps 下降到 2.5 Gbps 以下，激活传输延迟增加了 4.9 $\times$ （从 1.68 毫秒增加到 8.2 毫秒）。这使得静态管道设计中的通信延迟假设无效。我们的案例研究表明，缓解带宽争用可将端到端延迟减少 43%（图 2b）。

Cloud-only deployment limitations 在实际负载条件下出现。虽然纯云执行（云 GPU 上的所有层）实现了最佳的单请求延迟（LLaMA2-7B 为 179 毫秒），但它在并发负载下会发生灾难性的降级：在 QPS=5 下，由于单 GPU 序列化，排队占总延迟的 62%（478 毫秒中的 295 毫秒）。相比之下，边缘云管道协作（在第 12 层分割）通过管道并行性将排队时间减少到总延迟的 26%，尽管引入了网络传输开销，但仍实现了 39% 的端到端延迟降低和 64% 的吞吐量提高（图 3）。这说明边云协同的根本优势不在于资源的增加，而在于负载下管道并行的吞吐量倍增效应。

见解 1. *Dynamic resource contention at the edge, due to concurrent multi-model deployment and bandwidth competition, invalidates static partitioning assumptions. This necessitates adaptive edge-cloud orchestration that dynamically responds to real-time resource availability and workload characteristics.*



(a) Average latency across split points (b) Throughput across split points

图 3: 泊松到达下 LLaMA2-7B 的分割点 (SP) 研究 (QPS=3/4/5)。仅云 (SP=0) 和仅边缘 (SP=32) 始终不是最优的；中间分割点通过管道并行性实现最佳延迟-吞吐量权衡。

2.2 异构架构约束

边缘云架构表现出基本的资源不对称性，这对管道设计造成了结构性限制。这些三维差异——计算 (0.48 $\times$)、内存 (2.5 $\times$) 和通信 (80 $\times$)——通过跨系统层复合的级联效应从根本上限制了管道的适应性。

Computation imbalance 产生阶段延迟偏差，从而阻碍有效的管道平衡。H100 GPU (989 TFLOPS) 与边缘的 NVIDIA Thor (2070 TOPS) 反映了 0.48 $\times$ 的计算差异。在 LLaMA A2-70B 预填充期间，这种差异表现为不对称执行模式：注意力层在域之间表现出 2.7 $\times$ 延迟差异，而前馈网络表现出 1.8 $\times$ 差异。关键的见解是，不同的算子类型会经历不同程度的异质性放大——由于内存带宽限制，注意力操作在边缘设备上受到不成比例的影响，而嵌入层的性能下降却很小。这种特定于操作员的异构性会产生不规则的管道气泡，从而阻碍传统的负载平衡，当快速阶段等待异构性放大的慢速阶段时，会产生高达 67% 的理论空闲时间。

Memory hierarchy mismatch 通过限制阶段粒度来强制架构刚性。边缘 DRAM (64GB, 51.2 GB/s) 与云 HBM (140GB, 3.35 TB/s) 之间存在 65 $\times$ 带宽差异，加剧了内存容量限制。这种不匹配表现在两个关键方面：(1) *Stage fragmentation* - 边缘内存约束强制进行细粒度分区，从而减少计算通信重叠机会；(2) *Cache hierarchy violations* - 针对 HBM 优化的 KV 缓存访问模式在 DRAM 上表现出严重的性能下降，为长上下文推理造成 4.2 $\times$ 延迟损失。层次结构不匹配阻碍了灵活的阶段重新分配：针对 HBM 访问模式优化的阶段在没有基本算法重组的情况下无法有效迁移到基于 DRAM 的边缘节点。

Communication topology asymmetry 创建主导运行时特性的带宽-延迟权衡。边缘云链路 (10 Gbps, 15 毫秒 RTT) 与云互连 (800 Gbps RDMA, 1.5 s RTT) 表现出 80 $\times$ 带宽和 10,000 $\times$ 延迟差异。这种不对称性使得跨边缘云边界的张量并行性变得异常昂贵。

aries: AllReduce overhead increases from 0.8ms (NVLink) to 25ms (edge-cloud) for LLaMA2-70B attention, yielding 94% efficiency degradation. The topology asymmetry also creates *activation accumulation effects* where boundary stages buffer increasing intermediate results during network congestion, causing memory pressure cascades throughout the pipeline.

These constraints exhibit *multiplicative interaction effects* that severely limit pipeline adaptability. Computation imbalance amplifies memory pressure; memory constraints reduce communication overlap; communication latency prevents load redistribution. This creates a constraint satisfaction problem with limited feasible solutions, fundamentally restricting adaptability compared to homogeneous environments.

Insight 2. *Heterogeneous edge-cloud architectures create architectural rigidity through multiplicative constraint interactions—computation imbalance, memory hierarchy mismatches, and communication asymmetries—that severely limit pipeline adaptability.*

2.3 Performance Bottlenecks at Boundary Stage

Edge-cloud pipelines exhibit dynamic bottleneck migration that compounds the structural constraints from §2.2. Unlike homogeneous systems where bottlenecks remain localized, heterogeneous edge-cloud architectures create shifting performance constraints that invalidate static optimization approaches.

Edge node bottlenecks emerge from memory pressure amplified by concurrent workloads. When multiple LLM tasks compete for limited edge memory, cache eviction triggers pipeline stalls for KV reconstruction. Our measurements show 43% latency increases during peak load with four concurrent tasks. The computation imbalance (0.48× disparity) dynamically amplifies: attention-heavy workloads saturate edge compute while embedding operations underutilize cloud resources, creating asymmetric pipeline bubbles.

Boundary stage bottlenecks occur at edge-cloud transition points where structural asymmetries concentrate. These stages accumulate the compounded effects of computation imbalance, memory constraints, and communication latency. During LLaMA2-70B prefilling, boundary stages experience up to 82% idle time as edge-cloud execution mismatches are amplified by dynamic load variations. The boundary becomes a convergence point for multiple constraint violations, creating cascading pipeline stalls.

Network link bottlenecks arise from bandwidth contention that transforms the 80× communication disparity into a dynamic constraint. Concurrent tasks reduce effective bandwidth from 10 Gbps to 2.5 Gbps, increasing activation transmission latency from 1.68 ms to 8.2 ms (4.9× increase). This dynamic degradation invalidates static pipeline assumptions, as boundary stages must buffer increasing activation volumes while waiting for congested network transfers.

The critical insight is bottleneck migration: performance constraints shift dynamically among edge nodes, boundary stages, and network links based on workload characteristics

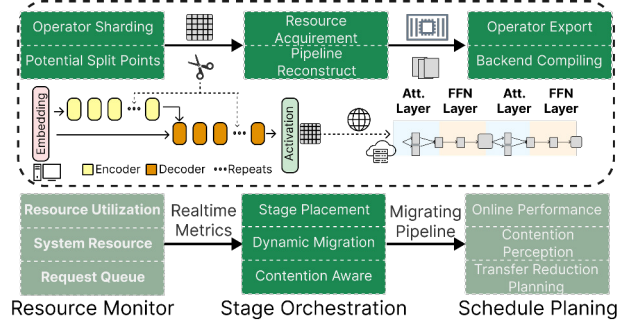


Fig. 4: DYNPIPE architecture: (1) Heterogeneous Pipeline Construction profiles operators and builds boundary-optimized pipelines offline; (2) Dynamic Orchestration monitors resource conditions and switches between pre-computed configurations; (3) Hierarchical State Management preserves KV cache continuity during reconfiguration through predictive staging.

and resource contention. This migration, combined with the architectural rigidity from three-dimensional constraints, prevents reactive adaptation strategies that assume fixed bottleneck locations.

Insight 3. *Performance bottlenecks in edge-cloud pipelines migrate dynamically across system components, amplified by structural constraints. This dynamic migration, combined with architectural rigidity, necessitates proactive multi-configuration orchestration rather than reactive adaptation.*

3. System Overview

DYNPIPE (Fig. 4) addresses the three challenges identified in §2 through three co-designed mechanisms.

Boundary-Aware Pipeline Construction (§4.1) formulates edge-cloud partitioning as a boundary-constrained optimization (Eq. 1) and uses dynamic programming to generate a compact portfolio of split-point configurations—each targeting a distinct resource regime (bandwidth-, compute-, or memory-constrained). A formal bound shows the portfolio size scales with the number of resource regimes rather than model depth, keeping it small in practice.

Proactive Multi-Configuration Orchestration (§4.2) selects among pre-computed configurations at runtime via the Latency-Regulated Placement (LRP) algorithm. LRP adapts a weighting parameter λ to the current bottleneck (network, compute, or memory), with hysteresis and cooldown mechanisms to prevent oscillatory switching while maintaining sub-millisecond decision latency.

Hierarchical State Management (§4.3) decomposes pipeline state into three tiers by migration criticality (KV caches, intermediate activations, auxiliary metadata) and pre-stages parameters through L1/L2/L3 memory hierarchy using predictive boundary estimation. When bandwidth drops below 1 Gbps, an adaptive recomputation fallback trades modest compute overhead for 90% bandwidth reduction, bounding worst-case migration to <120 ms.

aries: LLaMA2-70B 注意力的 AllReduce 开销从 0.8 毫秒（NVLink）增加到 25 毫秒（边缘云），导致效率下降 94%。拓扑不对称还会产生 *activation accumulation effects*，其中边界阶段在网络拥塞期间缓冲不断增加的中间结果，从而导致整个管道的内存压力级联。

这些约束表现出严重限制管道适应性的 *multiplicative interaction effects*。计算不平衡会放大内存压力；内存限制减少了通信重叠；通信延迟会阻止负载重新分配。这产生了可行解决方案有限的约束满足问题，从根本上限制了与同质环境相比的适应性。

见解 2. *Heterogeneous edge-cloud architectures create architectural rigidity through multiplicative constraint interactions—computation imbalance, memory hierarchy mismatches, and communication asymmetries—that severely limit pipeline adaptability.*

2.3 边界阶段的性能瓶颈

边缘云管道表现出动态瓶颈迁移，加剧了第 2.2 节中的结构约束。与瓶颈保持局部的同构系统不同，异构边缘云架构会产生不断变化的性能约束，从而使静态优化方法失效。

Edge node bottlenecks 并发工作负载放大的内存压力。当多个 LLM 任务竞争有限的边缘内存时，缓存驱逐会触发 KV 重建的管道停顿。我们的测量显示，在四个并发任务的峰值负载期间，延迟增加了 43%。计算不平衡（0.48× 差异）动态放大：注意力密集的工作负载使边缘计算饱和，而嵌入操作未充分利用云资源，从而产生不对称的管道泡沫。

Boundary stage bottlenecks 发生在结构不对称集中的边缘-云过渡点。这些阶段累积了计算不平衡、内存限制和通信延迟的复合效应。在 LLaMA2-70B 预填充期间，边界阶段的空闲时间高达 82%，因为边缘云执行不匹配会因动态负载变化而放大。该边界成为多个约束违规的汇聚点，从而产生级联管道停顿。

Network link bottlenecks 带宽争用将 80× 通信差异转化为动态约束。并发任务将有效带宽从 10 Gbps 减少到 2.5 Gbps，将激活传输延迟从 1.68 毫秒增加到 8.2 毫秒（增加 4.9×）。这种动态退化使静态管道假设失效，因为边界阶段必须在等待拥塞的网络传输时缓冲不断增加的激活量。

关键的洞察是瓶颈迁移：性能约束根据工作负载特征在边缘节点、边界阶段和网络链路之间动态转移

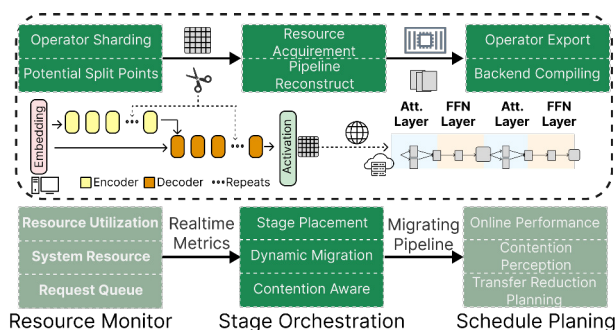


图 4: DynoPipe 架构：（1）异构管道构建对算子进行概要分析并离线构建边界优化的管道；（2）动态编排监控资源状况并在预先计算的配置之间切换；（3）分层状态管理通过预测分级在重新配置期间保持 KV 缓存连续性。

和资源争夺。这种迁移与三维约束的架构刚性相结合，防止了假设固定瓶颈位置的反应性适应策略。

见解 3. *Performance bottlenecks in edge-cloud pipelines migrate dynamically across system components, amplified by structural constraints. This dynamic migration, combined with architectural rigidity, necessitates proactive multi-configuration orchestration rather than reactive adaptation.*

三、系统概述

DynoPipe（图 4）通过三种共同设计的机制解决了第 2 节中确定的三个挑战。

边界感知管道构建（第 4.1 节）将边缘云分区制定为边界约束优化（方程 1），并使用动态编程生成紧凑的分割点配置组合 - 每个配置都针对不同的资源机制（带宽、计算或内存受限）。正式界限显示投资组合规模随资源体系的数量而不是模型深度而变化，从而在实践中保持较小的规模。

主动多配置编排（第 4.2 节）通过延迟调节布局 (LRP) 算法在运行时在预先计算的配置中进行选择。LRP 根据当前瓶颈（网络、计算或内存）调整权重参数 α ，并通过滞后和冷却机制来防止振荡切换，同时保持亚毫秒级决策延迟。

分层状态管理（第 4.3 节）通过迁移关键性（KV 缓存、中间激活、辅助元数据）将管道状态分解为三层，并使用预测边界估计通过 L1/L2/L3 内存层次结构预置参数。当带宽降至 1 Gbps 以下时，自适应重新计算回退会以适度的计算开销换取 90% 的带宽减少，将最坏情况的迁移限制在 <120 毫秒。

4. DYNPIPE Design

DYNPIPE addresses diverse edge scenarios through boundary-aware pipeline construction, hierarchical state management, and dynamic orchestration. By jointly optimizing computational and communication constraints, DYNPIPE enables adaptive inference across the edge-cloud continuum.

4.1 Edge-Cloud Collaborative Parallelism

Edge-cloud collaborative parallelism requires *boundary-aware* pipeline construction that explicitly models cross-domain transition costs. The core innovation centers on *boundary stages*—transition points where computation migrates between edge and cloud domains. Unlike homogeneous pipelines, boundary stages must reconcile computational asymmetry (0.48× disparity), memory hierarchy mismatch (65× bandwidth gap), and communication overhead (80× disparity), making boundary placement the primary performance determinant.

Boundary placement governs the privacy-efficiency trade-off: edge-heavy configurations preserve privacy but sacrifice performance, while cloud-aggressive offloading maximizes efficiency at privacy cost. This distinguishes collaborative parallelism from traditional homogeneous optimization that assumes uniform resources.

1) Heterogeneity-Aware Pipeline Formulation

We formulate collaborative parallelism as a boundary-constrained optimization problem that explicitly models heterogeneous resource transitions. Consider model stages $\{l_1, l_2, \dots, l_n\}$ distributed across edge domain $\mathcal{E}$ and cloud domain $\mathcal{C}$, where execution time $T_{exec}(l_i)$ varies dramatically based on resource domain assignment. The boundary stage at position l_B incurs additional heterogeneous transition overhead $T_{boundary}(l_B)$ encompassing cross-domain communication and state synchronization.

The optimization objective captures the essence of boundary-aware collaborative parallelism:

$$\min \left\{ \max_{1 \leq i \leq n} \{T_{exec}(l_i) + \mathbb{I}_{boundary}(l_i) \cdot T_{boundary}(l_i)\} \right\} \quad (1)$$

where $\mathbb{I}_{boundary}(l_i) = 1$ if stage l_i crosses the edge-cloud boundary, and 0 otherwise. This formulation explicitly accounts for heterogeneous transition overhead that dominates collaborative parallelism performance, distinguishing it from homogeneous pipeline optimization.

The boundary overhead $T_{boundary}(l_i)$ encompasses three heterogeneity-specific components: activation tensor serialization and deserialization, cross-domain transfer across heterogeneous network links, and state synchronization requirements between edge and cloud domains. Additionally, potential data format conversion between heterogeneous accelerators (e.g., CPU tensors to GPU format) contributes to the overall boundary transition cost.

Traditional pipeline optimization fails in edge-cloud environments due to resource asymmetry. Homogeneous approaches

assume uniform resources, but edge devices exhibit only 52% computational efficiency versus cloud GPUs. This imbalance creates cascading stalls, degrading throughput by up to 3.2× compared to optimal boundary placement.

2) Multi-Configuration Boundary Optimization

Traditional pipeline systems generate a single optimal configuration, but edge-cloud heterogeneity demands multiple boundary configurations to adapt to dynamic resource conditions. DYNPIPE constructs a portfolio of boundary configurations, each optimized for different heterogeneous scenarios: bandwidth-constrained configurations minimize activation transfers; compute-constrained configurations maximize cloud utilization; memory-constrained configurations respect edge device limitations.

The boundary optimization employs dynamic programming to efficiently explore the exponential search space of heterogeneous resource assignments. We define $F(d, l; \mathcal{R})$ as the optimal execution time for stages $[1, l]$ using d heterogeneous domains under resource constraint set $\mathcal{R}$:

$$F(d, l; \mathcal{R}) = \min_{1 \leq i \leq l} \left\{ \max \{F(d-1, i-1; \mathcal{R}), T_{exec}(i, l) + T_{boundary}(i)\} \mid \mathcal{R}_{constraint}(l) \right\} \quad (2)$$

This recurrence explicitly models boundary stage overhead when transitioning between heterogeneous domains, enabling precise optimization under resource asymmetry. The constraint set $\mathcal{R}_{constraint}(l)$ ensures that each configuration respects domain-specific limitations: edge memory bounds, cloud GPU availability, and network bandwidth thresholds.

The multi-configuration approach addresses static pipeline systems' inability to adapt to dynamic heterogeneous conditions. Edge-cloud environments exhibit temporal variations that shift optimal boundary placement within minutes: bandwidth fluctuations reduce capacity by 60-80% during peak usage, while thermal throttling decreases edge performance by 25-40%. Static configurations optimized for average conditions lead to resource underutilization or performance degradation under these dynamic scenarios.

3) Activation-Aware Boundary Selection

The boundary selection algorithm evaluates heterogeneous transition costs through three domain-specific criteria. Activation-minimal boundaries ($\min |A_b|$) target tensor compression points where attention sparsity or quantization reduces cross-domain transfer volumes. Computation-balanced boundaries distribute workload proportionally to device capabilities ($T_{edge}(1, b) \approx 0.48 \cdot T_{cloud}(b+1, n)$), reflecting empirically observed edge-cloud computational asymmetry. Memory-aware boundaries respect edge constraints ($\sum_{i=1}^b M_i \leq M_{edge}^{max}$) while maximizing local computation to minimize cloud dependency.

The algorithm integrates these criteria through adaptive weighting that responds to resource conditions: bandwidth contention prioritizes activation-minimal boundaries, compu-

4. DYNPIPE 设计

DynoPipe 通过边界感知管道构建、分层状态管理和动态编排解决不同的边缘场景。通过联合优化计算和通信约束，DynoPipe 实现了跨边缘云连续体的自适应推理。

4.1 边云协同并行

边缘-云协作并行性需要显式建模跨域转换成本的 *boundary-aware* 管道构建。核心创新集中在 *boundary stages* ——计算在边缘域和云域之间迁移的过渡点。与同质管道不同，边界阶段必须协调计算不对称（0.48×差异）、内存层次结构不匹配（65×带宽差距）和通信开销（80×差异），使边界布局成为主要性能决定因素。

边界放置控制着隐私与效率的权衡：边缘密集的配置可以保护隐私，但会牺牲性能，而云攻击性卸载则以隐私为代价最大化效率。这将协作并行性与假设统一资源的传统同质优化区分开来。

1) Heterogeneity-Aware Pipeline Formulation

我们将协作并行性表述为一个边界约束的优化问题，它显式地模拟异构资源转换。考虑模型阶段 $\{x_i\}_{i=1}^n$ ， x_i 分布在边缘域 $\mathcal{E}$ 和云域 $\mathcal{C}$ 上，其中执行时间 $x_{i,exec}$ 根据资源域分配而变化很大。位置 x_i 处的边界阶段会产生额外的异构转换开销 $x_{i,boundary}$ ，包括跨域通信和状态同步。

优化目标抓住了边界感知协作并行性的本质：

$$\min \left\{ \max_{1 \leq i \leq n} \{T_{exec}(l_i) + \mathbb{I}_{boundary}(l_i) \cdot T_{boundary}(l_i)\} \right\} \quad (1)$$

其中，如果阶段 x_i 穿过边云边界，则 $\mathbb{I}_{boundary}(x_i) = 1$ ，否则为 0。该公式明确考虑了主导协作并行性能的异构转换开销，将其与同构管道优化区分开来。

边界开销 $x_{i,boundary}$ 包含三个特定于异构性的组件：激活张量序列化和反序列化、跨异构网络链路的跨域传输以及边缘域和云域之间的状态同步要求。此外，异构加速器之间潜在的数据格式转换（例如，CPU 张量到 GPU 格式）也会增加总体边界转换成本。

由于资源不对称，传统的管道优化在边缘云环境中失败。同质方法

假设资源统一，但与云 GPU 相比，边缘设备的计算效率仅为 52%。这种不平衡会造成级联停顿，与最佳边界放置相比，吞吐量降低高达 3.2×。

2) Multi-Configuration Boundary Optimization

传统的管道系统生成单一的最佳配置，但边云异构性需要多个边界配置以适应动态资源条件。DynoPipe 构建了一系列边界配置，每种配置都针对不同的异构场景进行了优化：带宽受限的配置最大限度地减少了激活传输；计算受限的配置可最大限度地提高云利用率；内存受限配置尊重边缘设备限制。

边界优化采用动态规划来有效地探索异构资源分配的指数搜索空间。我们定义 $\{x_i\}_{i=1}^n$ 作为阶段 $[1, n]$ 的最佳执行时间，在资源约束集 $\mathcal{R}$ 下使用 x_i 异构域：

$$F(d, l; \mathcal{R}) = \min_{1 \leq i \leq l} \left\{ \max \{F(d-1, i-1; \mathcal{R}), T_{exec}(i, l) + T_{boundary}(i)\} \mid \mathcal{R}_{constraint}(l) \right\} \quad (2)$$

这种递归显式地模拟了异构域之间转换时的边界阶段开销，从而实现了资源不对称下的精确优化。约束集 $\mathcal{R}_{constraint}$ 确保每个配置都遵守特定于域的限制：边缘内存边界、云 GPU 可用性和网络带宽阈值。

多配置方法解决了静态管道系统无法适应动态异构条件的问题。边缘-云环境表现出时间变化，会在几分钟内改变最佳边界放置：带宽波动在高峰使用期间使容量减少 60-80%，而热节流使边缘性能降低 25-40%。针对平均条件优化的静态配置会导致在这些动态场景下资源利用不足或性能下降。

3) Activation-Aware Boundary Selection

边界选择算法通过三个特定领域的标准来评估异构转换成本。激活最小边界（ $\min |x_i|$ ）目标张量压缩点，其中注意力稀疏或量化减少了跨域传输量。计算平衡边界根据设备功能按比例分配工作负载（ $x_{i,dge}(1, x_i) \approx 0.48 \cdot x_{i,loud}(x_i + 1, x_i)$ ），反映了经验观察到的边缘云计算不对称性。内存感知边界尊重边缘约束（ $\sum_{i=1}^b x_i \leq x_{edge}^{max}$ ），同时最大化本地计算以最小化云依赖性。

该算法通过响应资源条件的自适应加权集成了这些标准：带宽争用优先考虑激活最小边界、计算

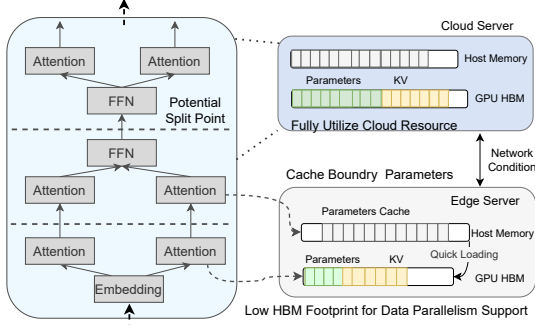


Fig. 5: Potential split points in the LLM computational graph.

tational stress emphasizes computation-balanced placement, and memory pressure favors memory-aware configurations. This eliminates exhaustive boundary search while maintaining optimal performance across heterogeneous scenarios.

4) Boundary Configuration Portfolio

DYNOPIPE pre-computes 3-5 boundary configurations targeting distinct heterogeneous scenarios (Fig. 5). The bandwidth-constrained configuration places boundaries after attention layers where tensor sparsity minimizes communication volume. The compute-optimized configuration leverages abundant cloud resources through early boundary placement, accepting larger activation transfers for superior throughput. The memory-constrained configuration optimizes for minimal edge footprint when memory pressure exceeds 90%.

The orchestration module monitors heterogeneous resource conditions through lightweight telemetry, switching configurations when performance thresholds are violated. This proactive approach avoids the high reconstruction overhead of reactive reconfiguration while maintaining service availability under varying heterogeneous conditions.

Portfolio Size Bound. For a single edge-cloud boundary (§4.2), the DP recurrence (Eq. 1) maps each resource state $\mathcal{R} = (r_{bw}, r_{comp}, r_{mem})$ to an optimal split point $b^*(\mathcal{R}) \in \{0, 1, \dots, n\}$, where n is the number of candidate split points (model-dependent; equal to the number of transformer blocks for decoder-only LLMs). Because T_{exec} and $T_{boundary}$ vary continuously with resource conditions while b^* is discrete, $b^*(\mathcal{R})$ is piecewise-constant: the resource space partitions into contiguous cells that each map to a single optimal SP. With k resource dimensions and at most m qualitatively distinct regimes per dimension (e.g. free / moderate / contention for bandwidth), the number of cells is at most m^k . Monotonic relationships between resource constraints and boundary placement—higher bandwidth favors earlier SP (more cloud utilization), higher memory pressure favors later SP (smaller edge footprint)—collapse many cells to the same b^* . The set of distinct optimal configurations therefore satisfies $|K| \leq \min(n, m^k)$. For architectures with uniform transformer blocks (e.g. LLaMA), per-layer cost functions are near-identical, so the monotonicity collapse is strong and $|K| \ll n$ in practice. For models with

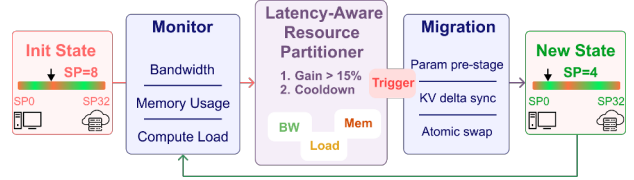


Fig. 6: DYNOPIPE's dynamic boundary optimization decision flow. The system monitors resource conditions to select optimal split points from pre-computed configurations, triggering reconfiguration when performance thresholds are violated.

heterogeneous layer types (e.g. MoE, mixed vision-language encoders), per-layer cost variation may increase $|K|$, but the bound still grows with resource-regime count m^k rather than linearly with model depth n .

Residual Connections and Boundary Placement. Modern LLMs (e.g., LLaMA) employ residual connections *within* each transformer block ($x_{out} = x_{in} + \text{Attn}(x_{in}) + \text{FFN}(\cdot)$), but these residuals resolve at block boundaries. DYNOPIPE places split points exclusively *between* complete transformer blocks (Fig. 5), so the output tensor at each candidate boundary is a fully resolved hidden state with no outstanding skip connections. This eliminates cross-domain residual synchronization and keeps the activation tensor transmitted to the cloud identical in shape and semantics regardless of placement, requiring no additional buffering or recomputation.

Privacy Risk Analysis. Achieving full privacy requires either edge-only deployment (limited by device memory) or homomorphic encryption (with impractical latency [30–32]). DYNOPIPE takes a practical approach: sensitive embeddings stay on the edge, and only deeper activations are sent to the cloud. While intermediate representations could leak partial input data in theory [10], effective attacks are rare in real deployments given the need for detailed model knowledge and representative data [33]. Using multi-boundary splits (locally computing both input and output layers) may slightly improve privacy, but doubles edge work and network cost without eliminating the residual risk from intermediate activations [10].

4.2 Dynamic Orchestration of Stages

Dynamic orchestration enables adaptive boundary placement in edge-cloud environments facing fluctuating resources—such as network bandwidth drops during congestion, computational load imbalance, and up to 10× variation in activation sizes across model layers due to structural sparsity (Fig. 6). The key insight is that LLMs have heterogeneous communication cost: attention layers benefit from structured sparsity [34, 35] and quantization yields varying compression ratios [11, 36]. Communication-efficient layers appear at different locations in the model, creating trade-offs between transfer reduction and edge compute load. Systematic orchestration is thus required to allocate stages dynamically across edge and cloud, optimizing overall performance under changing conditions (Fig. 7).

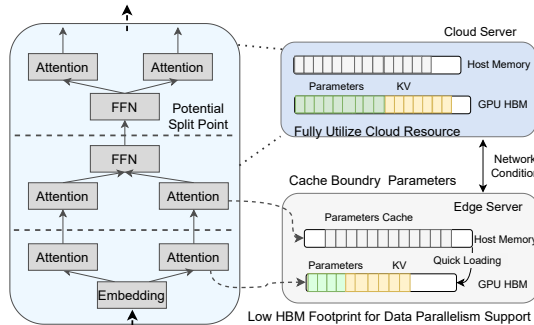


图 5: LLM 计算图中的潜在分点。

国家压力强调计算平衡的放置，而内存压力则有利于内存感知配置。这消除了详尽的边界搜索，同时在异构场景中保持最佳性能。

4) Boundary Configuration Portfolio

DynoPipe 针对不同的异构场景预先计算 3-5 个边界配置（图 5）。带宽受限的配置将边界放置在注意层之后，其中张量稀疏性最小化了通信量。计算优化的配置通过早期边界放置利用丰富的云资源，接受更大的激活传输以获得卓越的吞吐量。当内存压力超过 90% 时，内存受限配置会优化以最小化边缘占用空间。

编排模块通过轻量级遥测来监控异构资源状况，并在违反性能阈值时切换配置。这种主动方法避免了反应性重新配置的高重建开销，同时在不同的异构条件下保持服务可用性。

投资组合规模有限。对于单个边-云边界（第 4.2 节），DP 递归（方程 1）将每个资源状态 $\mathcal{R} = (\mathcal{R}_v, \mathcal{R}_{cmp}, \mathcal{R}_{em})$ 映射到最佳分割点 $\mathcal{S}(\mathcal{R}) \in \{0, 1, \dots, \mathcal{N}\}$ ，其中 $\mathcal{N}$ 是候选分割点的数量（取决于模型；等于仅解码器 LLM 的转换器块的数量）。由于 $\mathcal{R}_{xec}$ 和 $\mathcal{R}_{boundary}$ 随资源条件连续变化，而 $\mathcal{S}$ 是离散的，因此 $\mathcal{S}(\mathcal{R})$ 是分段常数：资源空间划分为连续的单元，每个单元映射到单个最优 SP。凭借 $\mathcal{S}$ 资源维度和每个维度至多 $\mathcal{N}$ 定性不同的机制（例如免费/中等/争用带宽），小区数量至多为 $\mathcal{N}^3$ 。资源约束和边界放置之间的单调关系——较高的带宽有利于较早的 SP（更多的云利用率），较高的内存压力有利于较晚的 SP（较小的边缘占用空间）——将许多单元折叠到相同的 $\mathcal{S}$ 。因此，一组不同的最佳配置满足 $|\mathcal{S}| \leq \min(\mathcal{N}, \mathcal{N})$ 。对于具有统一变压器块的架构（例如 LLaMA），每层成本函数几乎相同，因此单调性崩溃很强，并且在实践中是 $|\mathcal{S}| \ll \mathcal{N}$ 。对于型号

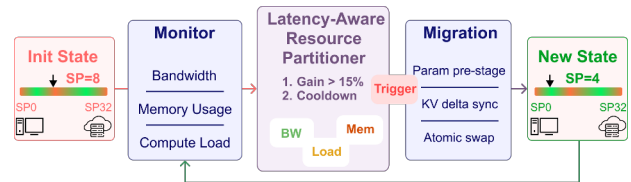


图 6: DynoPipe 的动态边界优化决策流程。系统监控资源状况，从预先计算的配置中选择最佳分割点，在违反性能阈值时触发重新配置。

异构层类型（例如 MoE、混合视觉语言编码器），每层成本变化可能会增加 $|\mathcal{S}|$ ，但界限仍然随着资源机制计数 $\mathcal{N}$ 增长，而不是与模型深度 $\mathcal{N}$ 线性增长。

残余连接和边界放置。现代 LLM（例如 LLaMA）在每个变压器块 $(\mathcal{R}_u = \mathcal{R}_v + \text{Attn}(\mathcal{R}_v) + \text{FFN}(\cdot))$ 采用残差连接 *within*，但这些残差在块边界处解析。DynoPipe 仅将分割点放置在 *between* 完整的转换器块中（图 5），因此每个候选边界处的输出张量是完全解析的隐藏状态，没有突出的跳过连接。这消除了跨域残差同步，并保持传输到云的激活张量在形状和语义上相同，无论放置如何，不需要额外的缓冲或重新计算。

隐私风险分析。实现完全隐私需要仅边缘部署（受设备内存限制）或同态加密（具有不切实际的延迟[30-32]）。DynoPipe 采用实用方法：敏感嵌入保留在边缘，只有更深层次的激活才会发送到云端。虽然理论上中间表示可能会泄漏部分输入数据[10]，但考虑到需要详细的模型知识和代表性数据[33]，在实际部署中有效的攻击很少见。使用多边界分割（本地计算输入和输出层）可能会稍微改善隐私，但会使边缘工作和网络成本加倍，而不会消除中间激活的残留风险[10]。

4.2 阶段的动态编排

动态编排可在面临资源波动的边缘云环境中实现自适应边界放置，例如拥塞期间网络带宽下降、计算负载不平衡以及由于结构稀疏而导致跨模型层的激活大小最多 10x 变化（图 6）。关键的见解是 LLM 具有异构通信成本：注意力层受益于结构化稀疏性[34, 35]，量化产生不同的压缩比[11, 36]。通信高效层出现在模型中的不同位置，在减少传输和边缘计算负载之间进行权衡。因此，需要系统编排来跨边缘和云动态分配阶段，从而在不断变化的条件下优化整体性能（图 7）。

1) Adaptive Boundary Selection Framework

The dynamic orchestration problem constitutes a constrained optimization challenge that must simultaneously balance performance, communication overhead, and resource utilization while respecting computational graph dependencies. We formulate this as a boundary-constrained placement optimization where the objective minimizes the maximum communication time across pipeline stages:

$$\min \left\{ \max_{1 \leq i \leq D} \left\{ \sum_{1 \leq j \leq G_i} T_{req}(i, j) + \lambda \cdot T_{comm}(i, j) \right\} \right\} \quad (3)$$

where $T_{req}(i, j)$ represents the computational time for processing request j on device i , $T_{comm}(i, j)$ captures the communication overhead for stage transitions, and λ is a dynamic weighting factor that adapts to current network conditions. The inner summation aggregates computational and communication costs across all stages G_i assigned to device i , while the outer maximization identifies the critical bottleneck device.

To manage the exponential complexity of this optimization, we leverage domain-specific constraints inherent to edge-cloud architectures. Since edge-to-cloud communication latency significantly exceeds intra-cloud communication (typically 10-50 $\times$ higher), we constrain the system to a single edge-to-cloud boundary per request. This transforms the complex multi-boundary optimization into an efficient single split-point selection problem: stages before the boundary execute at the edge, while subsequent stages run in the cloud. Exact runtime solving of Eq. 3 is too costly: evaluating all n split points incurs $O(n)$ overhead, which exceeds the sub-millisecond latency budget needed for online serving. Instead, we use a heuristic portfolio-selection approach, narrowing each decision to $O(|K|)$ table lookups ($|K| \leq 5$), balancing efficiency and coverage of practical deployment regimes (§5.6).

The boundary selection framework operates through three interconnected optimization criteria:

Communication-Minimal Boundaries: Identify split points that minimize activation transfer volume by exploiting attention sparsity, quantization compression, and layer-specific activation patterns. These boundaries target natural compression points where $|A_b| \ll |A_{avg}|$, reducing cross-domain transfer costs.

Load-Balanced Boundaries: Distribute computational workload proportionally to device capabilities, ensuring $T_{edge}(1, b) \approx \alpha \cdot T_{cloud}(b+1, n)$ where α reflects the empirically observed edge-cloud computational asymmetry (0.4-0.6).

Resource-Constrained Boundaries: Respect edge device limitations while maximizing local computation, ensuring $\sum_{i=1}^b M_i \leq M_{edge}^{max}$ and $\sum_{i=1}^b C_i \leq C_{edge}^{available}$.

2) Real-Time Boundary Adaptation Algorithm

Runtime Objective. At each decision epoch, LRP selects a boundary $b \in \mathcal{K}$ from the pre-profiled portfolio that minimizes predicted end-to-end latency $\hat{T}_{ec}(b)$ —comprising queueing, edge execution, network transfer, cloud execution, and switching overhead—under current resource conditions, rather

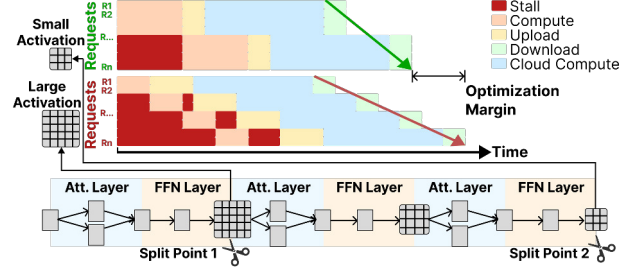


Fig. 7: DYNOPipe mitigates network bandwidth contention by dynamically selecting activation volumes transmitted over the access network, achieving progressively better optimization as request volume increases.

than re-solving the full placement problem online. The Latency-Regulated Placement (LRP) algorithm dynamically reconfigures pipeline boundaries through pre-computed configuration portfolios, addressing the trade-off between adaptation responsiveness and computational overhead.

LRP employs bottleneck-aware adaptive weighting that adjusts placement priorities based on the dominant system bottleneck. When bandwidth is limited, λ increases to favor activation-minimizing boundaries; under heavy computational load, λ decreases for balanced workload distribution; in memory-constrained situations, device memory limits take precedence. This context-sensitive adjustment maintains low latency while ensuring sub-millisecond decision overhead.

Configuration stability uses hysteresis mechanisms preventing oscillatory switching. Boundary transitions occur only when performance improvements exceed threshold δ (15-20%), with cooldown periods preventing rapid reconfigurations. This reduces reconfiguration overhead from seconds to milliseconds.

The pre-computed portfolio maintains 3-5 specialized configurations: bandwidth-constrained (boundaries after attention layers), compute-constrained (early cloud placement), memory-constrained (minimal edge footprint), and balanced (proportional distribution). This achieves competitive performance with sub-millisecond selection latency.

Blended Constraints and Sensitivity. Under simultaneous resource constraints (bandwidth and memory pressure), the `SelectBoundary` function (Algorithm 1) evaluates active triggers and selects the configuration minimizing worst-case stage latency across all constraints. The hysteresis threshold (15-20%) and cooldown period balance adaptation responsiveness with stability: aggressive switching increases reconfiguration overhead with minimal gains, while conservative thresholds delay beneficial adaptations. The weighting parameter λ uses exponential smoothing to prevent oscillation under fluctuating network conditions.

Assumptions, Rationale, and Limitations. LRP adopts a finite-portfolio heuristic rather than exhaustive online re-optimization or a learned controller: exhaustive search adds $O(n)$ per-decision overhead incompatible with sub-millisecond budgets, while learned policies require training data and are hard to stabilize under shifting edge conditions. The heuristic

1) Adaptive Boundary Selection Framework

动态编排问题构成了一个受限的优化挑战，必须同时平衡性能、通信开销和资源利用率，同时尊重计算图依赖性。我们将其表述为边界约束布局优化，其目标是最小化跨管道阶段的最大通信时间：

$$\min \left\{ \max_{1 \leq i \leq D} \left\{ \sum_{1 \leq j \leq G_i} T_{req}(i, j) + \lambda \cdot T_{comm}(i, j) \right\} \right\} \quad (3)$$

其中 $T_{eq}(i, j)$ 表示设备上处理请求 i 的计算时间， $T_{comm}(i, j)$ 表示阶段转换的通信开销， λ 是适应当前网络条件的动态权重因子。内部求和汇总了分配给设备 i 的所有阶段 j 的计算和通信成本，而外部最大化则确定了关键瓶颈设备。

为了管理这种优化的指数复杂性，我们利用边缘云架构固有的特定领域约束。由于边缘到云通信延迟显著超过云内通信（通常高 10-50x），因此我们将系统每个请求限制在单个边缘到云边界。这将复杂的多边界优化转化为高效的单分割点选择问题：边界之前的阶段在边缘执行，而后续阶段在云端运行。方程的精确运行时求解成本太高：评估所有分割点会产生高昂开销，这超出了在线服务所需的亚毫秒延迟预算。相反，我们使用启发式组合选择方法，将每个决策缩小到 (i, j) 表查找 ($|i| \leq 5$)，平衡实际部署制度的效率和覆盖范围 (§5.6)。

边界选择框架通过三个相互关联的优化标准运行：

通信最小边界：通过利用注意力稀疏性、量化压缩和特定于层的激活模式来识别最小化激活传输量的分割点。这些边界以自然压缩点为目标，其中 $|v_i| \ll |v_{vg}|$ ，从而降低跨域传输成本。

负载平衡边界：根据设备功能按比例分配计算工作负载，确保 $d_{ge}(1, i) \approx d_{loud}(i, 1)$ ，其中 i 反映了经验观察到的边缘云计算不对称性 (0.4-0.6)。

资源受限边界：尊重边缘设备限制，同时最大化本地计算，确保 $\sum_{i=1}^b v_i \leq v_{edge}^{max}$ 和 $\sum_{i=1}^b v_i \leq v_{edge}^{available}$ 。

2) Real-Time Boundary Adaptation Algorithm

运行时目标。在每个决策时期，LRP 从预先分析的投资组合中选择一个边界 $i \in \mathcal{K}$ ，该边界最大限度地减少了预测的端到端延迟 $T_{ec}(i)$ （包括排队、边缘执行、网络传输、云执行和切换开销），而不是在当前资源条件下

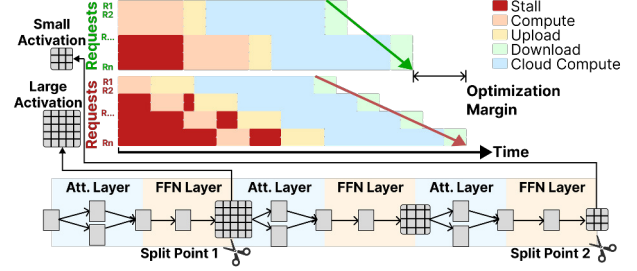


图 7: DynoPipe 通过动态选择通过接入网络传输的激活量来缓解网络带宽争用，随着请求量的增加逐步实现更好的优化。

而不是在线重新解决完整的放置问题。延迟调节布局 (LRP) 算法通过预先计算的配置组合动态地重新配置管道边界，解决自适应响应性和计算开销之间的权衡。

LRP 采用瓶颈感知自适应权重，根据主要系统瓶颈调整放置优先级。当带宽有限时， λ 会增加以支持激活 - 最小化边界；在繁重的计算负载下， λ 减小以实现平衡的工作负载分配；在内存受限的情况下，设备内存限制优先。这种上下文相关的调整可保持低延迟，同时确保亚毫秒级决策开销。

配置稳定性使用滞后机制来防止振荡切换。仅当性能改进超过阈值 λ (15-20%) 时才会发生边界转换，并且冷却期会阻止快速重新配置。这将重新配置开销从几秒减少到几毫秒。

预先计算的组合维护 3-5 个专门的配置：带宽约束（注意层之后的边界）、计算约束（早期云放置）、内存约束（最小边缘占用）和平衡（比例分布）。这实现了亚毫秒级选择延迟的竞争性能。

混合约束和敏感性。在同时资源约束（带宽和内存压力）的情况下，SelectBoundary 函数（算法 1）评估活动触发器并选择配置，以在所有约束下最小化最坏情况阶段延迟。滞后阈值 (15-20%) 和冷却时间平衡适应响应性与稳定性：积极的切换以最小的增益增加重新配置开销，而保守的阈值会延迟有益的适应。权重参数 λ 使用指数平滑来防止网络波动条件下的振荡。

假设、基本原理和限制。LRP 采用有限投资组合启发式方法，而不是详尽的在线重新优化或学习控制器：详尽的搜索增加了与亚毫秒级预算不兼容的每个决策开销，而学习策略需要训练数据，并且在变化的边缘条件下很难稳定。启发式

assumes (1) a single edge-cloud boundary per request, valid when cross-domain latency dominates intra-domain communication (10–50× in our testbed); (2) monotonic resource-performance relationships, confirmed for uniform transformer architectures (Table III) but potentially weaker for MoE or mixed-modality models; and (3) representative offline profiles. The trigger ordering (bandwidth $\rightarrow$ compute $\rightarrow$ memory) reflects empirical bottleneck severity; under blended constraints, `SelectBoundary` minimizes worst-case stage latency across all active triggers. LRP does not guarantee global optimality—architectures with irregular per-layer costs or highly non-stationary environments may require larger portfolios or learned adaptation.

Profiling Methodology. T_{comp} , T_{mem} , and T_{comm} in Algorithm 1 are obtained from a lightweight offline profiling phase that executes representative prompts (128 tokens, batch=1/4/8) on each device pair. Per-layer execution time and activation sizes are measured and stored in a lookup table (<30 KB per model); runtime adaptation uses these profiles plus live telemetry (bandwidth, GPU utilization, memory pressure sampled every 500 ms) to re-evaluate placement decisions.

Four-Dimensional Optimization Relationship. The ablation study in §5.6 reveals that optimal split-point placement is a dynamic function of four interacting variables: request arrival rate λ , network conditions (RTT, bandwidth), edge compute capability α_{edge} , and model structure. The total request latency decomposes as:

$$T_{\text{total}} = T_{\text{queue}}(\lambda, \mu) + T_{\text{edge}}(\text{SP}) + T_{\text{net}}(\text{SP}, \text{RTT}) + T_{\text{cloud}}(\text{SP}) \quad (4)$$

where the effective pipeline service rate $\mu(\text{SP}) = 1/\max(T_{\text{edge}}(\text{SP}), T_{\text{cloud}}(\text{SP}))$ determines system capacity. At low load ($\lambda \ll \mu$), minimizing single-request latency favors $\text{SP}=0$ (cloud-only). As load approaches capacity ($\lambda \rightarrow \mu$), maximizing throughput through balanced split points that satisfy $T_{\text{edge}} \approx T_{\text{cloud}}$ becomes critical. Under network contention, the optimal SP shifts to balance transmission volume against pipeline efficiency. This relationship validates DYNPIPE’s multi-configuration portfolio: different configurations naturally correspond to different operating regions in this four-dimensional space, and the LRP algorithm selects the appropriate configuration based on real-time monitoring of all four dimensions.

4.3 State Orchestration in Heterogeneous Pipelines

Hierarchical State Decomposition and Predictive Staging. DYNPIPE introduces a state decomposition framework that categorizes pipeline state into three distinct tiers based on migration criticality and reconstruction complexity. *Critical frontier states* encompass KV caches and attention weights essential for maintaining generation coherence across boundary transitions. *Intermediate activations* represent layer outputs that can be selectively recomputed with bounded overhead. *Auxiliary metadata* includes normalization statistics and positional encodings that exhibit high spatial locality.

DYNPIPE employs predictive parameter staging that exploits

Algorithm 1: Latency-Regulated Placement (LRP)

Input: Model stages $S_1, \dots, S_n$; Edge devices E ; Cloud devices C ; Network state N

Result: Heuristic stage placement minimizing pipeline latency

```

1 Function ComputeCost ( $S_i, D_j, \lambda$ ):
2   return  $\alpha \cdot T_{\text{comp}}(S_i, D_j) + \beta \cdot T_{\text{mem}}(S_i, D_j) + \lambda \cdot T_{\text{comm}}(S_i)$ ;
3 Function SelectBoundary ( $\text{system\_state}$ ):
4   if bandwidth <  $\tau_{\text{bw}}$  then
5     return boundaryactivation_minimal
6   end
7   if edge_load >  $\tau_{\text{compute}}$  then
8     return boundaryearly_cloud
9   end
10  if memory_pressure >  $\tau_{\text{mem}}$  then
11    return boundarymemory_aware
12  end
13  return boundarybalanced;
14 Pre-compute boundary configurations for resource scenarios;
15 while System active do
16   state = monitor network, compute, memory conditions;
17   target_boundary = SelectBoundary (state);
18   if Boundary change AND improvement >  $\delta$  then
19      $\lambda$  = adapt weighting to dominant bottleneck;
20     foreach stage  $S_i$  do
21       Find device  $D^*$  minimizing ComputeCost ( $S_i, D_j$ ,
22          $\lambda$ );
23       Assign  $S_i \rightarrow D^*$  with hysteresis threshold  $\delta$ ;
24     end
25     Execute boundary migration;
26 end
```

reconfiguration locality through a three-tier memory hierarchy. L1 staging caches hot boundary configurations with sub-millisecond access; L2 maintains secondary transition candidates; L3 provides demand-driven fallback. This anticipatory positioning eliminates cold-start penalties by pre-staging critical parameters at likely transition boundaries, reducing migration overhead from seconds to microseconds.

Single- vs. Multi-User KV Cache Management: Under single-user serving, KV caches grow linearly with context length and reside entirely within the assigned domain (edge or cloud). Under multi-user concurrent serving, per-request KV caches compete for edge GPU memory, which may trigger earlier boundary shifts to cloud-heavy configurations that reduce edge memory pressure. DYNPIPE’s state orchestration handles both regimes uniformly: the hierarchical tier structure evicts cold per-request caches (LRU policy) while preserving hot caches for active requests, and boundary selection accounts for aggregate KV memory via the memory-pressure trigger (>90%) in Algorithm 1.

Adaptive Recomputation Scheduling: When uplink bandwidth drops below 1Gbps, DYNPIPE dynamically switches to a recomputation-based recovery mode. The system maintains lightweight checkpoints at strategic layer boundaries and reconstructs intermediate states through selective forward passes, trading modest compute overhead (15-25% increase) for dramatic bandwidth reduction (90% decrease).

假设 (1) 每个请求有一个边缘-云边界，当跨域延迟主导域内通信时有效（在我们的测试台中为 10-50 $\times$ ）；(2) 单调的资源-性能关系，在统一变压器架构中得到证实（表 III），但在 MoE 或混合模态模型中可能较弱；(3) 有代表性的离线资料。触发器排序（带宽 $\rightarrow$ 计算 $\rightarrow$ 内存）反映了经验瓶颈严重性；在混合约束下，SelectBoundary 最大限度地减少所有活动触发器的最坏情况阶段延迟。LRP 不能保证全局最优性——每层成本不规则或高度不稳定的环境的架构可能需要更大的投资组合或学习适应。

分析方法。算法 1 中的 $\bar{\alpha}_{comp}$ 、 $\bar{\alpha}_{mem}$ 和 $\bar{\alpha}_{comm}$ 是从轻量级离线分析阶段获得的，该阶段在每个设备上执行代表性提示（128 个令牌，批次=1/4/8）。测量每层执行时间和激活大小并将其存储在查找表中（<每个模型 30 KB）；运行时适应使用这些配置文件加上实时遥测（带宽、GPU 利用率、每 500 毫秒采样一次的内存压力）来重新评估放置决策。

四维优化关系。第 5.6 节中的消融研究表明，最佳分割点放置是四个相互作用变量的动态函数：请求到达率 λ 、网络条件（RTT、带宽）、边缘计算能力 $\bar{\alpha}_{edge}$ 和模型结构。总请求延迟分解为：

$$T_{total} = T_{queue}(\lambda, \mu) + T_{edge}(SP) + T_{net}(SP, RTT) + T_{cloud}(SP) \quad (4)$$

其中有效管道服务率 $\bar{\alpha}(SP) = 1/\max(\bar{\alpha}_{dge}(SP), \bar{\alpha}_{loud}(SP))$ 决定系统容量。在低负载 ($\lambda \ll \bar{\alpha}$) 下，最大限度地减少单个请求延迟有利于 $SP=0$ （仅限云）。当负载接近容量 ($\lambda \rightarrow \bar{\alpha}$) 时，通过满足 $\bar{\alpha}_{dge} \approx \bar{\alpha}_{loud}$ 的平衡分割点来最大化吞吐量变得至关重要。在网络竞争的情况下，最佳 SP 会转向平衡传输量与管道效率。这种关系验证了 DynoPipe 的多配置组合：不同的配置自然对应于这个四维空间中的不同操作区域，LRP 算法基于对所有四个维度的实时监控来选择合适的配置。

4.3 异构管道中的状态编排

分层状态分解和预测分期。DynoPipe 引入了一个状态分解框架，该框架根据迁移关键性和重建复杂性将管道状态分为三个不同的层。Critical frontier states 包含 KV 缓存和注意力权重，这对于维持跨边界转换的生成一致性至关重要。Intermediate activations 表示可以以有限的开销选择性地重新计算的层输出。Auxiliary metadata 包括表现出高空间局部性的归一化统计数据和位置编码。

DynoPipe 采用预测参数分级，利用

Algorithm 1: Latency-Regulated Placement (LRP)

Input: Model stages $S_1, \dots, S_n$; Edge devices E ; Cloud devices C ; Network state N

Result: Heuristic stage placement minimizing pipeline latency

```

1 Function ComputeCost ( $S_i, D_j, \lambda$ ):
2   return  $\alpha \cdot T_{comp}(S_i, D_j) + \beta \cdot T_{mem}(S_i, D_j) + \lambda \cdot T_{comm}(S_i)$ ;
3 Function SelectBoundary ( $system\_state$ ):
4   if bandwidth <  $\tau_{bw}$  then
5     return boundaryactivation_minimal
6   end
7   if edge_load >  $\tau_{compute}$  then
8     return boundaryearly_cloud
9   end
10  if memory_pressure >  $\tau_{mem}$  then
11    return boundarymemory_aware
12  end
13  return boundarybalanced;
14 Pre-compute boundary configurations for resource scenarios;
15 while System active do
16   state = monitor network, compute, memory conditions;
17   target_boundary = SelectBoundary (state);
18   if Boundary change AND improvement >  $\delta$  then
19      $\lambda$  = adapt weighting to dominant bottleneck;
20     foreach stage  $S_i$  do
21       Find device  $D^*$  minimizing ComputeCost ( $S_i, D_j, \lambda$ );
22       Assign  $S_i \rightarrow D^*$  with hysteresis threshold  $\delta$ ;
23     end
24     Execute boundary migration;
25   end
26 end

```

通过三层内存层次结构重新配置局部性。L1 暂存以亚毫秒级访问缓存热边界配置；L2 维护次要过渡候选；L3 提供需求驱动的回退。这种预期定位通过在可能的转换边界预先设置关键参数来消除冷启动损失，从而将迁移开销从几秒减少到微秒。

单用户与多用户 KV 缓存管理：在单用户服务下，KV 缓存随上下文长度线性增长，并完全驻留在指定的域（边缘或云）内。在多用户并发服务下，每个请求的 KV 缓存都会竞争边缘 GPU 内存，这可能会触发早期边界转移到云密集型配置，从而减少边缘内存压力。DynoPipe 的状态编排统一处理这两种机制：层次结构逐出每个请求的冷缓存（LRU 策略），同时保留活动请求的热缓存，边界选择通过算法 1 中的内存压力触发器 (>90%) 考虑聚合 KV 内存。

自适应重新计算调度：当上行链路带宽降至 1Gbps 以下时，DynoPipe 动态切换到基于重新计算的恢复模式。该系统在战略层边界维护轻量级检查点，并通过选择性前向传递重建中间状态，以适度的计算开销（增加 15-25%）换取带宽的大幅减少（减少 90%）。

5. Evaluation

5.1 Experimental Setup

We evaluate DYNOPipe on a production-representative testbed that mirrors real-world edge-cloud deployments, using real request traces to stress-test dynamic orchestration under realistic workload variability. The testbed comprises a cloud cluster and a geographically distributed edge tier. The cloud cluster consists of 12 servers equipped with 16 NVIDIA A40 GPUs (48GB HBM2 each), interconnected via 100 Gbps RDMA-capable InfiniBand for low-latency inter-GPU communication. Each cloud server runs Ubuntu 22.04 with 512GB DDR4 memory, providing sufficient host resources for batched request processing and state management.

Edge Environment. The edge tier consists of NVIDIA RTX 3090 GPUs (24GB GDDR6X) deployed at geographically distributed edge locations. Edge devices connect to the cloud cluster through a shared 10 Gbps uplink (measured RTT: 5–50ms depending on cross-region routing and congestion), reflecting realistic WAN conditions for edge deployments. Edge-to-edge communication uses a local 1 Gbps LAN (RTT <3ms).

Workload. We evaluate DYNOPipe using the Microsoft Azure Functions (MAF) trace [37], which captures production-scale request patterns with coefficient of variation exceeding 2.5 and inter-arrival times spanning four orders of magnitude. This workload’s inherent burstiness and temporal clustering stress-tests dynamic resource allocation mechanisms critical for edge-cloud LLM serving [8, 38–41]. We map MAF invocations to LLM inference requests while preserving temporal correlations, with request parameters (prompt length, output tokens, temperature) sampled from the Splitwise distribution [42] to reflect realistic generation workloads.

Baselines. We benchmark DYNOPipe against FlexNN [43] and EdgeShard [27], representing distinct paradigms in distributed LLM inference. FlexNN uses edge-centric pipeline parallelism with static layer allocation, exposing memory-compute constraints in edge-only deployments. EdgeShard implements fixed edge-cloud partitioning with offline optimization, revealing penalties of static allocation under dynamic conditions. We include a **Cloud-only** baseline (SP=0) to isolate pure-cloud performance and demonstrate that DYNOPipe’s gains arise from pipeline parallelism rather than cloud augmentation. These baselines isolate DYNOPipe’s dynamic orchestration contributions.

Evaluation Metrics. We evaluate DYNOPipe using standard LLM serving metrics [44]: time to first token (TTFT), token throughput (Tput), and end-to-end (E2E) latency (mean, P50, P99) across batched requests.

5.2 System Performance

Controlled workload evaluation (Table I) exposes fundamental scalability limits in edge-only architectures. FlexNN suffers catastrophic degradation as load increases: TTFT latency increases sharply from 19.68s to 68.53s between 2-8 QPS, with memory exhaustion rendering it unusable beyond 8

TABLE I: Performance of DYNOPipe and FlexNN in LLaMA3.1-8B.

QPS	Queue (s)		TTFT (s)		Tput (tok/s)	
	DYNOPipe	FlexNN	DYNOPipe	FlexNN	DYNOPipe	FlexNN
2	0.30	43.86	0.10	19.68	922	277
4	0.40	55.78	0.47	27.03	1118	291
8	0.91	110.06	0.74	68.53	1327	304
16	1.25	-	1.56	-	1857	-
24	1.49	-	2.56	-	2471	-

QPS: query/second.

Config: Temp=0.7, max out=4096 toks, max in=16392 toks, timeout=5s.

TABLE II: Performance in different scale of models.

Model	Act.(M)	Type	Throughput		TPOT (ms)	
			FlexNN	DYNOPipe	FlexNN	DYNOPipe
Whisper-V2	0.32	Speech	1545	3900	30	36
LLaMA2-7B	1.2	Chat	263	2645	54	73
LLaMA3.1-8B	1.45	Chat	290	1857	68	65
Qwen-14B-VL	2.1	MM	-	1640	-	84
Qwen-32B-VL	4.8	MM	-	1309	-	134
LLaMA3-70B	8.2	MM	-	830	-	160

TPOT (Time Per Output Token) is calculated as the average decode time. MM refers to the multimodal model using LLM as the backbone. - indicates that the model cannot be deployed on the edge due to memory constraints.

QPS. DYNOPipe maintains sub-second performance throughout, delivering 98.5% TTFT reduction and 4.4× throughput gains.

The root cause is edge memory-compute mismatch: efficient batching demands longer contexts, yet memory constraints force choosing between context length and concurrency. DYNOPipe breaks this constraint through strategic layer placement—retaining input embeddings and initial attention at the edge while offloading memory-intensive operations to cloud. This enables 128K+ context processing with 16+ concurrent requests on resource-constrained edge hardware.

Scalability advantages amplify with model complexity (Table II). While FlexNN cannot deploy beyond 8B parameters, DYNOPipe serves 70B models at 830 tokens/second with 160ms per-token latency. Performance gains scale with activation complexity: from 2.5× throughput improvement for WhisperV2 to 10.1× for LLaMA2-7B, demonstrating DYNOPipe’s fundamental advantage of edge-cloud workload distribution.

TPOT vs. Throughput Trade-off. The apparent contradiction between higher TPOT and lower mean latency in Table II arises from the distinction between per-token decode time and end-to-end request latency. DYNOPipe’s pipeline parallelism introduces modest per-token overhead from edge-to-cloud traversal (network transfer + cross-domain synchronization), increasing TPOT from 54ms to 73ms for LLaMA2-7B. However, this pipeline structure enables concurrent request processing that dramatically reduces queueing delay—the dominant latency component under load. Pipeline parallelism overlaps edge and cloud stages, allowing multiple simultaneous requests and reducing waiting time far more than per-token overhead adds. The 10.1× throughput gain reflects concurrent processing across pipeline stages, while TPOT measures sequential per-token decode latency. Detailed latency breakdown appears in §5.6.

5. 评价

5.1 实验设置

我们在反映现实世界边缘云部署的生产代表性测试台上评估 DynoPipe，使用真实请求跟踪在实际工作负载变化下对动态编排进行压力测试。该测试床包括一个云集群和一个地理上分布的边缘层。该云集群由 12 台服务器组成，配备 16 个 NVIDIA A40 GPU（每个 48GB HBM2），通过支持 100 Gbps RDMA 的 InfiniBand 互连，以实现低延迟的 GPU 间通信。每台云服务器运行 Ubuntu 22.04，配备 512GB DDR4 内存，为批量请求处理和状态管理提供充足的主机资源。

边缘环境。边缘层由部署在地理分布的边缘位置的 NVIDIA RTX 3090 GPU (24GB GDDR6X) 组成。边缘设备通过共享的 10 Gbps 上行链路连接到云集群（测量的 RTT：5-50 毫秒，具体取决于跨区域路由和拥塞），反映了边缘部署的实际 WAN 条件。边缘到边缘通信使用本地 1 Gbps LAN (RTT <3ms)。

工作量。我们使用 Microsoft Azure Functions (MAF) 跟踪 [37] 评估 DynoPipe，该跟踪捕获变化系数超过 2.5 且到达间隔时间跨越四个数量级的生产规模请求模式。该工作负载固有的突发性和时间集群压力测试了对于边缘云 LLM 服务至关重要的动态资源分配机制 [8, 38–41]。我们将 MAF 调用映射到 LLM 推理请求，同时保留时间相关性，并使用从 Splitwise 分布 [42] 中采样的请求参数（提示长度、输出标记、温度）来反映实际的生成工作负载。

基线。我们将 DynoPipe 与 FlexNN [43] 和 EdgeShard [27] 进行基准测试，代表分布式 LLM 推理中的不同范例。FlexNN 使用以边缘为中心的管道并行性和静态层分配，暴露了仅边缘部署中的内存计算约束。EdgeShard 通过离线优化实现固定边云分区，揭示动态条件下静态分配的惩罚。我们包含纯云基线 (SP=0) 来隔离纯云性能，并证明 DynoPipe 的收益来自管道并行性而不是云增强。这些基线隔离了 DynoPipe 的动态编排贡献。

评估指标。我们使用标准 LLM 服务指标 [44] 评估 DynoPipe：第一个令牌的时间 (TTFT)、令牌吞吐量 (Tput) 和批量请求的端到端 (E2E) 延迟（平均值、P50、P99）。

5.2 系统性能

受控工作负载评估（表 I）暴露了仅边缘架构中的基本可扩展性限制。随着负载的增加，FlexNN 会遭受灾难性的退化：在 2-8 QPS 之间，TTFT 延迟从 19.68 秒急剧增加到 68.53 秒，内存耗尽导致超过 8 秒后无法使用

表 I: DynoPipe 和 FlexNN 在 LLaMA3.1-8B 中的性能。

QPS	Queue (s)		TTFT (s)		Tput (tok/s)	
	DynoPipe	FlexNN	DynoPipe	FlexNN	DynoPipe	FlexNN
2	0.30	43.86	0.10	19.68	922	277
4	0.40	55.78	0.47	27.03	1118	291
8	0.91	110.06	0.74	68.53	1327	304
16	1.25	-	1.56	-	1857	-
24	1.49	-	2.56	-	2471	-

QPS: query/second.
Config: Temp=0.7, max out=4096 toks, max in=16392 toks, timeout=5s.

表二：不同规模模型的性能。

Model	Act.(M)	Type	Throughput		TPOT (ms)	
			FlexNN	DynoPipe	FlexNN	DynoPipe
Whisper-V2	0.32	Speech	1545	3900	30	36
LLaMA2-7B	1.2	Chat	263	2645	54	73
LLaMA3.1-8B	1.45	Chat	290	1857	68	65
Qwen-14B-VL	2.1	MM	-	1640	-	84
Qwen-32B-VL	4.8	MM	-	1309	-	134
LLaMA3-70B	8.2	MM	-	830	-	160

TPOT（每个输出令牌的时间）计算为平均解码时间。MM 是指以 LLM 为骨干的多模态模型。- 表示由于内存限制，模型无法部署在边缘。

QPS。DynoPipe 自始至终保持亚秒级性能，TTFT 减少 98.5%，吞吐量提高 4.4×。

根本原因是边缘内存计算不匹配：高效的批处理需要更长的上下文，但内存限制迫使在上下文长度和并发性之间进行选择。DynoPipe 通过战略层放置打破了这一限制——在边缘保留输入嵌入和初始注意力，同时将内存密集型操作卸载到云端。这可以在资源受限的边缘硬件上实现 128K + 上下文处理和 16+ 并发请求。

可扩展性优势随着模型复杂性的增加而放大（表 II）。虽然 FlexNN 无法部署超过 8B 参数，但 DynoPipe 可以以 830 个令牌/秒的速度提供 70B 模型，每个令牌延迟为 160 毫秒。性能增益随着激活复杂性的增加而增加：从 WhisperV2 的 2.5× 吞吐量提高到 LLaMA2-7B 的 10.1×，展示了 DynoPipe 在边缘云工作负载分配方面的基本优势。

TPOT 与吞吐量的权衡。表 II 中较高的 TPOT 和较低的平均延迟之间的明显矛盾源于每个令牌解码时间和端到端请求延迟之间的区别。DynoPipe 的管道并行性从边缘到云遍历（网络传输 + 跨域同步）引入了适度的每个令牌开销，将 LLaMA2-7B 的 TPOT 从 54 毫秒增加到 73 毫秒。然而，这种管道结构支持并发请求处理，从而显着减少排队延迟（负载下的主要延迟组件）。管道并行性与边缘和云阶段重叠，允许同时发出多个请求，减少的等待时间远远超过每个令牌增加的开销。10.1× 吞吐量增益反映了跨管道阶段的并发处理，而 TPOT 则测量每个令牌的顺序解码延迟。详细的延迟细分显示在第 5.6 节中。

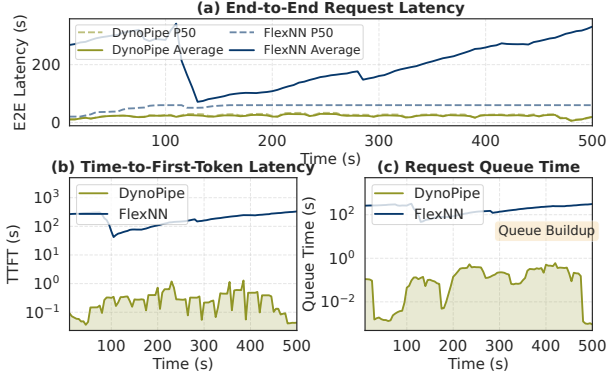


Fig. 8: Latency comparison of DYNOPiPE and FlexNN in LLaMA2.1-8B model under MAF workload. (a) End-to-end latency. (b) Time to First Token Latency. (c) Queue latency.

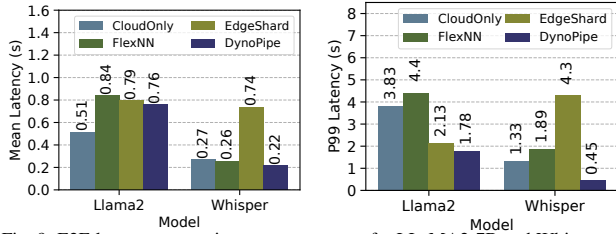


Fig. 9: E2E latency comparison across systems for LLaMA2-7B and Whisper-v2 models under the MAF workload. DYNOPiPE provides the lowest tail latency across both models and the best or near-best average latency.

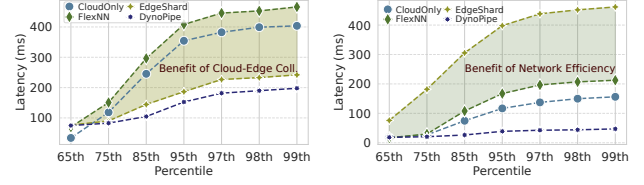
5.3 Real-World Workload Performance

We evaluate DYNOPiPE under the production MAF trace using LLaMA2-7B and Whisper-v2, and compare against CloudOnly, FlexNN, and EdgeShard [27]. CloudOnly runs the full model in the cloud, FlexNN keeps execution on the edge, EdgeShard uses a static split, and DYNOPiPE adapts the split online.

As shown in Fig. 9, DYNOPiPE mainly improves robustness under bursty arrivals. For LLaMA2-7B, it reduces P99 latency by 54%, 60%, and 16% compared with CloudOnly, FlexNN, and EdgeShard, while keeping mean latency close to the best baseline. For Whisper-v2, DYNOPiPE achieves the best overall performance, lowering mean latency by 19%, 15%, and 70%, and P99 latency by 66%, 76%, and 90%, respectively. These results show that online split adaptation is most beneficial for controlling tail latency under realistic edge-cloud workloads.

5.4 Latency Distribution and Tail Performance Analysis

We analyze DYNOPiPE’s latency characteristics under the MAF workload to understand its tail performance behavior—a critical metric for production deployments. The latency distribution analysis (Fig. 10) reveals fundamental architectural differences. Edge-only deployment (FlexNN) exhibits catastrophic tail latency degradation—latency increases 6.7× from the 65th to 99th percentile for LLaMA2-7B due to resource exhaustion. EdgeShard’s static partitioning provides marginal improvement (2.9× increase) but remains vulnerable to load spikes.



(a) Quantile of Latency of LLaMA2 (b) Quantile of Latency of Whisper
Fig. 10: Quantile of latency of LLaMA2 and Whisper in MAF workload. (a). LLaMA2-7B. (b). Whisper. The shaded areas respectively show the performance improvements brought by edge-cloud collaborative inference and communication reduction.

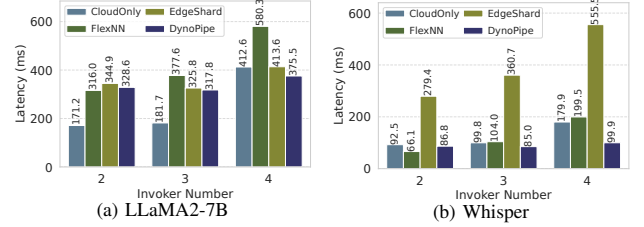


Fig. 11: Average end-to-end latency under concurrent multi-task execution as the number of invokers increases from 2 to 4. (a) LLaMA2-7B. (b) Whisper-v2.

DYNOPiPE achieves superior tail latency control with only 2.2× variation across quantiles through dynamic load balancing and adaptive pipeline partitioning. For Whisper, DYNOPiPE achieves 99.8% lower P99 latency than EdgeShard and 85.3% lower than FlexNN. These results demonstrate that DYNOPiPE’s adaptive optimization provides predictable performance guarantees essential for production systems, where tail latency directly impacts user satisfaction and service reliability.

5.5 Multi-Task Performance

We evaluate DYNOPiPE under concurrent multi-task execution by increasing the number of invokers from 2 to 4, comparing with CloudOnly, FlexNN, and EdgeShard on the same MAF-based workload. As shown in Fig. 11, the advantage of DYNOPiPE grows as concurrency rises. For LLaMA2-7B, CloudOnly has the lowest latency with 2–3 invokers, but DYNOPiPE outperforms others at 4 invokers and stays much more stable than FlexNN and EdgeShard as contention increases. For Whisper-v2, DYNOPiPE has the lowest latency from 3 invokers onward, while EdgeShard degrades sharply and both CloudOnly and FlexNN slow down at higher concurrency. These results show that dynamic boundary adaptation is most effective in multi-task scenarios: while it may not always yield the lowest latency at light load, it prevents the severe degradation seen in static or single-domain baselines under higher contention.

Multi-Tenant Implications. The multi-task evaluation demonstrates multi-tenant behavior: independent Poisson request streams from heterogeneous edge devices create concurrent resource contention analogous to multi-tenant serving. DYNOPiPE maintains <1.1× degradation with 4 concurrent sources and stable throughput across QPS=3/4/5, while EdgeShard degrades 45× and FlexNN degrades 1.8–3.1× under the same conditions. For context, production LLM serving systems such

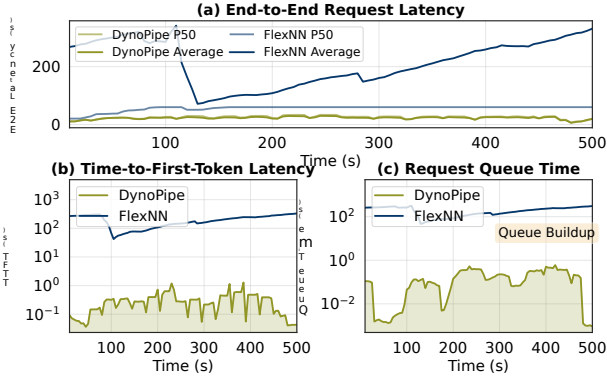


图 8: MAF 工作负载下 LLaMA3.1-8B 模型中 DynoPipe 和 FlexNN 的延迟比较。(a) 端到端延迟。(b) 首次令牌延迟时间。(c) 队列延迟。

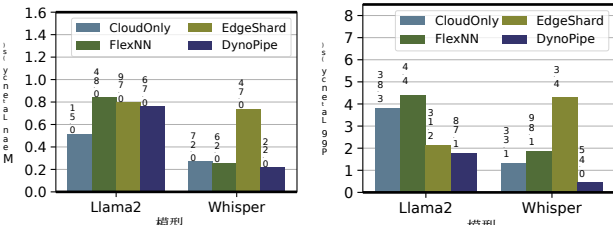


图 9: MAF 工作负载下 LLaMA2-7B 和 Whisper-v2 模型的跨系统 E2E 延迟比较。DynoPipe 在两个模型中提供最低的尾部延迟以及最佳或接近最佳的平均延迟。

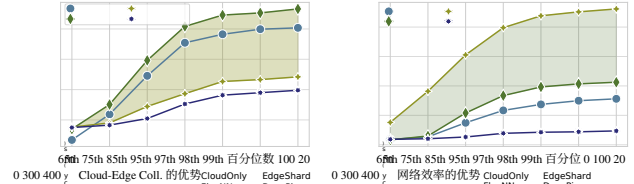
5.3 实际工作负载性能

我们使用 LLaMA2-7B 和 Whisper-v2 在生产 MAF 跟踪下评估 DynoPipe，并与 CloudOnly、FlexNN 和 EdgeShard [27] 进行比较。CloudOnly 在云中运行完整模型，FlexNN 在边缘保持执行，EdgeShard 使用静态分割，DynoPipe 在线调整分割。

如图9所示，DynoPipe主要提高突发到达情况下的鲁棒性。对于 LLaMA2-7B，与 CloudOnly、FlexNN 和 EdgeShard 相比，P99 延迟减少了 54%、60% 和 16%，同时保持平均延迟接近最佳基线。对于 Whisper-v2，DynoPipe 实现了最佳的整体性能，平均延迟分别降低了 19%、15% 和 70%，P99 延迟分别降低了 66%、76% 和 90%。这些结果表明，在线分割自适应对于控制实际边缘云工作负载下的尾延迟最为有利。

5.4 延迟分布和尾部性能分析

我们分析 DynoPipe 在 MAF 工作负载下的延迟特征，以了解其尾部性能行为——这是生产部署的关键指标。延迟分布分析（图 10）揭示了基本的架构差异。仅边缘部署（FlexNN）表现出灾难性的尾部延迟下降——由于资源耗尽，LLaMA2-7B 的延迟从第 65 个百分点增加到第 99 个百分点，增加了 6.7 $\times$ 。EdgeShard 的静态分区提供了边际改进（2.9 $\times$ 增加），但仍然容易受到负载峰值的影响。



(a) LLaMA2 的延迟分位数 (b) Whisper 的延迟分位数 图 10: MAF 工作负载中 LLaMA2 和 Whisper 的延迟分位数。(一个)。LLaMA2-7B。(二)。耳语。阴影区域分别表示边云协同推理和减少通信带来的性能提升。

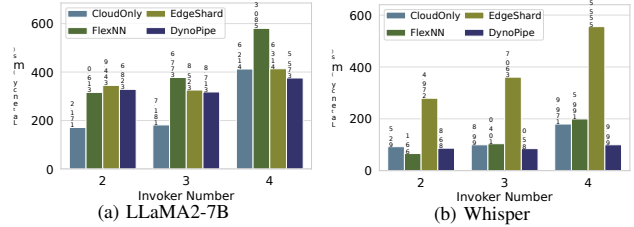


图 11: 当调用者数量从 2 个增加到 4 个时，并发多任务执行下的平均端到端延迟。(a) LLaMA2-7B。(b) 耳语-V2。

DynoPipe 通过动态负载平衡和自适应管道分区实现了卓越的尾部延迟控制，跨分位数仅有 2.2 $\times$ 变化。对于 Whisper，DynoPipe 的 P99 延迟比 EdgeShard 低 99.8%，比 FlexNN 低 85.3%。这些结果表明，DynoPipe 的自适应优化提供了对生产系统至关重要的可预测性能保证，其中尾部延迟直接影响用户满意度和服务可靠性。

5.5 多任务性能

我们通过将调用程序数量从 2 个增加到 4 个来评估并发多任务执行下的 DynoPipe，并与相同基于 MAF 的工作负载上的 CloudOnly、FlexNN 和 EdgeShard 进行比较。如图 11 所示，DynoPipe 的优势随着并发量的增加而增强。对于 LLaMA2-7B，CloudOnly 在 2-3 个调用程序时具有最低的延迟，但 DynoPipe 在 4 个调用程序时优于其他产品，并且随着争用的增加，保持比 FlexNN 和 EdgeShard 稳定得多。对于 Whisper-V2，DynoPipe 从 3 个调用者开始具有最低的延迟，而 EdgeShard 性能急剧下降，CloudOnly 和 FlexNN 在较高并发性下速度都会变慢。这些结果表明，动态边界适应在多任务场景中最有效：虽然它可能并不总是在轻负载下产生最低延迟，但它可以防止在较高争用情况下静态或单域基线中出现的严重退化。

多租户的影响。多任务评估演示了多租户行为：来自异构边缘设备的独立泊松请求流会创建类似于多租户服务的并发资源争用。DynoPipe 在 4 个并发源的情况下保持 <1.1 $\times$ 降级，并在 QPS=3/4/5 范围内保持稳定的吞吐量，而在相同条件下，EdgeShard 降级 45 $\times$ ，FlexNN 降级 1.8–3.1 $\times$ 。对于上下文，生产大语言模型服务系统，例如

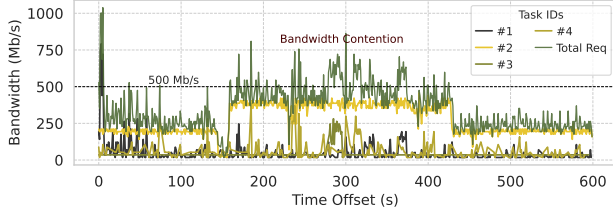


Fig. 12: Bandwidth usage of each tasks and the total bandwidth request in MAF workload of EdgeShard. The requests between different tasks exhibit burstiness, and there is bandwidth contention in overlapping peak periods, while the bandwidth utilization is low during the rest of the time.

as AlphaServe [41] report 1.2–1.5 $\times$ latency inflation under comparable multiplexing ratios on *homogeneous* GPU clusters; DYNOPipe achieves tighter degradation despite operating across *heterogeneous* edge-cloud domains with WAN latency variability. The low switching frequency (<2 reconfigurations/min at QPS=5) further confirms that dynamic boundary adaptation generalizes to multi-tenant workloads without oscillation or per-tenant starvation.

5.6 Case Study: Understanding Split-Point Dynamics

We conduct an ablation study on LLaMA2-7B (32 layers), comparing 9 split points (SP=0, 4, 8, ..., 32), 3 network regimes (Free: 5ms RTT; Moderate: 20ms; Contention: 50ms, Pareto jitter), and several load levels (QPS=3/4/5) using Poisson arrivals (128 requests per run). The edge GPU is 30% slower than the cloud, reflecting typical heterogeneity. **Cloud-only** (SP=0) runs all layers on the cloud GPU, with no pipelining or edge-cloud overlap. All baselines share the same model and workload; only split-point placement changes.

1) Cloud-Only vs. Collaborative Performance

Fig. 3 compares end-to-end latency and throughput across split points under increasing load. At QPS=5 with unconstrained network, cloud-only execution (SP=0) achieves 478ms average latency and 2.09 requests/second (rps) throughput, while the optimal collaborative configuration (SP=12) achieves 292ms latency and 3.43 rps—a 39% latency reduction and 64% throughput improvement over cloud-only.

The mechanism is pipeline parallelism: cloud-only (SP=0) has a mean service time of 179 ms (~5.6 rps capacity), so at QPS=5 the server operates at 89% utilization where Poisson arrivals and service-time variance cause nonlinear queueing buildup. SP=12 splits execution into concurrent edge (96 ms) and cloud (112 ms) stages, raising effective capacity to ~8.9 rps and reducing utilization to 56%, which is why collaborative execution substantially reduces queueing.

Edge-only execution (SP=32) is catastrophic under load: with 30% edge GPU slowdown, each request takes 256ms on the slower edge device, yielding capacity of only 3.9 rps. At QPS=5, the system is severely overloaded, producing 4498ms average latency—15.4 $\times$ worse than SP=12.

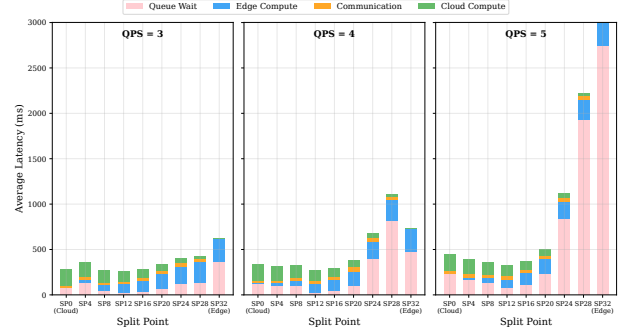


Fig. 13: Latency breakdown by component (queue/edge/network/cloud) across split points at QPS=5, Network Free. Queueing dominates cloud-only (SP=0) and edge-only (SP=32); collaborative split points minimize total latency by balancing pipeline stages.

TABLE III: Optimal split point and latency (ms) under varying load and network conditions for LLaMA2-7B.

QPS	Net. Free		Net. Moderate		Net. Contention	
	SP	Lat.	SP	Lat.	SP	Lat.
3	4	246	12	254	4	274
4	12	252	12	261	16	315
5	12	292	12	308	8	372

2) Latency Breakdown Analysis

Fig. 13 decomposes end-to-end latency into queueing, edge computation, network transfer, and cloud computation, revealing DYNOPipe’s performance gains. At QPS=5, cloud-only (SP=0) suffers 295ms queueing (62% of 478ms total), while SP=12 reduces queueing to 76ms (26% of 292ms total). The 186ms improvement stems primarily from pipeline parallelism nearly doubling system capacity, despite modest edge-cloud traversal overhead (7ms network + 96ms edge vs. 178ms cloud-only). This result also explains why cloud-only degrades sharply near QPS=5: once the single-GPU server approaches saturation, bursty arrivals and service-time variability quickly translate into queue buildup.

This explains the TPOT trade-off in Table II: while per-token decode time increases slightly, queueing reduction from 62% to 26% of total latency produces substantially better end-to-end performance under realistic load.

3) Dynamic Split-Point Necessity

The optimal split point shifts with both load intensity and network conditions (Fig. 14), validating the need for dynamic boundary adaptation. Table III shows that no single static split point is universally optimal: a static SP=12 incurs 36% degradation under Network Contention at QPS=4 (428ms vs. optimal SP=16 at 315ms), while a static SP=4 is 82% worse than optimal at QPS=5 (532ms vs. 292ms). These results validate DYNOPipe’s multi-configuration portfolio: the 3–5 pre-computed configurations correspond to distinct operating regimes, and a single offline-optimal boundary can suffer up to 82% penalty when conditions shift, justifying dynamic portfolio-based selection.

Portfolio Saturation Analysis. We verify the bound from

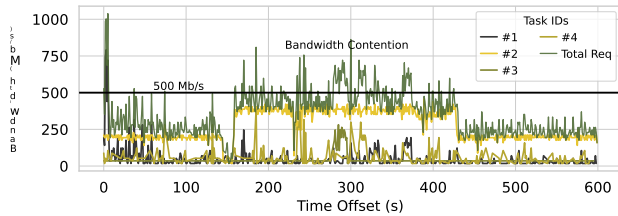


图 12: EdgeShard 的 MAF 工作负载中每个任务的带宽使用情况和总带宽请求。不同任务之间的请求呈现突发性，在重叠的高峰时段存在带宽争用，而其余时间带宽利用率较低。

AlpaServe [41] 报告称，在 *homogeneous* GPU 集群上，在可比较的复用率下，延迟膨胀为 1.2–1.5 $\times$ ；尽管跨 *heterogeneous* 边缘云域运行并具有 WAN 延迟变化，但 DynoPipe 仍实现了更严格的降级。低切换频率（QPS=5 下的 < 2 次重新配置/分钟）进一步证实动态边界自适应可推广到多租户工作负载，而不会出现振荡或每租户饥饿。

5.6 案例研究：理解分裂点动力学

我们对 LLaMA2-7B（32 层）进行消融研究，使用泊松到达（128 每次运行的请求数）。边缘 GPU 比云慢 30%，反映了典型的异构性。仅云（SP=0）在云 GPU 上运行所有层，没有管道或边缘云重叠。所有基线共享相同的模型和工作负载；仅分割点位置发生变化。

1) Cloud-Only vs. Collaborative Performance

图 3 比较了负载增加情况下分割点的端到端延迟和吞吐量。在网络不受约束的 QPS=5 下，纯云执行（SP=0）实现了 478 毫秒的平均延迟和 2.09 个请求/秒（rps）吞吐量，而最佳协作配置（SP=12）实现了 292 毫秒的延迟和 3.43 rps，与纯云相比，延迟减少了 39%，吞吐量提高了 64%。

该机制是管道并行性：仅云（SP=0）的平均服务时间为 179 毫秒（~5.6 rps 容量），因此在 QPS=5 下，服务器以 89% 的利用率运行，其中泊松到达和服务时间方差导致非线性排队累积。SP=12 将执行分为并发边缘（96 毫秒）和云端（112 毫秒）阶段，将有效容量提高到~8.9 rps，并将利用率降低到 56%，这就是协作执行大幅减少排队的原因。

仅边缘执行（SP=32）在负载下是灾难性的：边缘 GPU 减速 30%，每个请求在较慢的边缘设备上需要 256 毫秒，产生的容量仅为 3.9 rps。在 QPS=5 时，系统严重过载，产生 4498ms 的平均延迟，比 SP=12 差 15.4 $\times$ 。

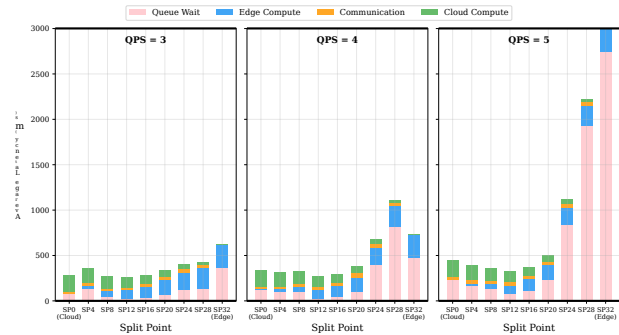


图 13: QPS=5、无网络的分割点上按组件（队列/边缘/网络/云）划分的延迟细分。队列在纯云（SP=0）和仅边缘（SP=32）中占主导地位；协作分割点通过平衡管道阶段来最大限度地减少总延迟。

表 III: LLaMA2-7B 在不同负载和网络条件下的最佳分割点和延迟（毫秒）。

QPS	Net. Free		Net. Moderate		Net. Contention	
	SP	Lat.	SP	Lat.	SP	Lat.
3	4	246	12	254	4	274
4	12	252	12	261	16	315
5	12	292	12	308	8	372

2) Latency Breakdown Analysis

图 13 将端到端延迟分解为排队、边缘计算、网络传输和云计算，揭示了 DynoPipe 的性能提升。At QPS=5, cloud-only (SP=0) suffers 295ms queueing (62% of 478ms total), while SP=12 reduces queueing to 76ms (26% of 292ms total). 186 毫秒的改进主要源于管道并行性几乎使系统容量翻倍，尽管边缘云遍历开销不大（7 毫秒网络 + 96 毫秒边缘 vs. 178 毫秒仅云）。这一结果还解释了为什么纯云在 QPS=5 附近急剧下降：一旦单 GPU 服务器接近饱和，突发性到达和服务时间变化很快就会转化为队列堆积。

这解释了表 II 中的 TPOT 权衡：虽然每个令牌的解码时间略有增加，但队列延迟从总延迟的 62% 减少到 26%，在实际负载下产生了更好的端到端性能。

3) Dynamic Split-Point Necessity

最佳分割点随着负载强度和网络条件的变化而变化（图 14），验证了动态边界自适应的需要。表 III 显示，没有一个静态分割点是普遍最佳的：静态 SP=12 在 QPS=4 时的网络争用情况下会导致 36% 的性能下降（428ms 与最佳 SP=16 在 315ms 时相比），而静态 SP=4 在 QPS=5 时比最佳状态差 82%（532ms 与 292ms）。这些结果验证了 DynoPipe 的多配置组合：3-5 个预先计算的配置对应于不同的操作方案，并且当条件发生变化时，单个离线最优边界可能会遭受高达 82% 的惩罚，这证明了基于动态组合的选择的合理性。

投资组合饱和度分析。我们验证边界

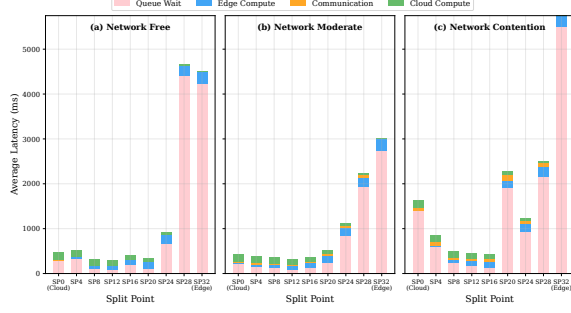


Fig. 14: Split-point performance under three network regimes at QPS=5. Optimal split point shifts from SP=12 (Network Free) to SP=8 (Network Contention), demonstrating the necessity of dynamic boundary adaptation.

§4.1 empirically: across the 9 operating conditions in Table III (3 load levels $\times$ 3 network regimes), only 4 distinct optimal split points emerge ($\{4, 8, 12, 16\}$) out of 32 candidate layers. A portfolio of size 2 (e.g. $\{4, 12\}$) misses SP=16 at QPS=4 under contention (315 ms optimal vs. 428 ms with SP=12, a 36% penalty) and SP=8 at QPS=5 under contention. A portfolio of size 4 covers all observed optima with zero residual gap; adding a fifth configuration yields no further improvement across all tested conditions. This is consistent with the theoretical prediction: for uniform transformer blocks, the monotonicity collapse is strong enough that $|K| = 4 \ll n = 32$. Models with heterogeneous layer structures may require larger portfolios, but the growth remains bounded by m^k rather than n .

4) Heterogeneous Edge Impact

Under realistic hardware heterogeneity with 30% edge GPU throughput reduction (Fig. 15), SP=12 achieves 215ms latency versus 179ms for cloud-only (SP=0), while edge-only (SP=32) takes 255ms—42% slower as the throughput penalty accumulates across all layers. Under load, however, the edge throughput disadvantage is dominated by the queuing benefit of pipeline parallelism. At QPS=5, the pipeline service rate at SP=12 ($\mu \approx 8.9$ rps) far exceeds both cloud-only ($\mu \approx 5.6$ rps) and edge-only ($\mu \approx 3.9$ rps), making the per-layer edge overhead a minor factor compared to the 64% throughput gain from collaboration. This validates DYNPIPE’s computation-balanced boundary selection: the LRP algorithm accounts for device-specific throughput asymmetry when computing optimal boundaries, ensuring the pipeline bottleneck ($\max(T_{\text{edge}}, T_{\text{cloud}})$) is minimized despite hardware heterogeneity.

Sensitivity to Heuristic Choices. We verify LRP parameter robustness with the multi-regime ablation and portfolio analysis above. Portfolios with fewer than four candidates lose adaptability (e.g., missing SP=16 leads to 36% penalty at QPS=4; see Table III), while more than four brings no extra benefit, consistent with the $|K| \leq \min(n, m^k)$ bound. A hysteresis threshold $\delta \in [15\%, 20\%]$ strikes a good balance: lowering it causes excessive switching for minor fluctuations, while raising it only slightly delays necessary changes. Overall, these results across nine operating conditions show that the finite-portfolio approach is robust in realistic edge-cloud scenarios.

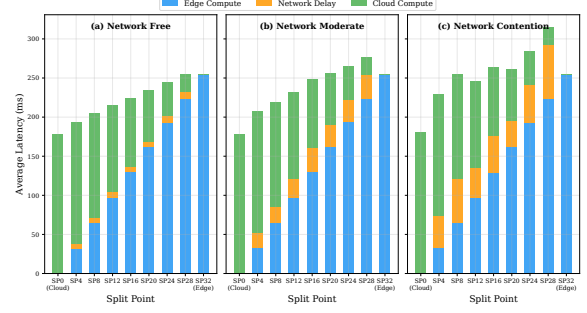


Fig. 15: Latency breakdown with heterogeneous edge GPU (30% slowdown). Edge computation penalty accumulates with more layers assigned to edge, making cloud-heavy split points more efficient for single requests while pipeline parallelism favors balanced splits under load.

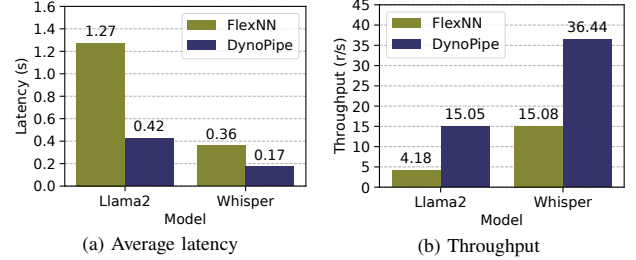


Fig. 16: Performance comparison of DYNPIPE and FlexNN in edge-only deployment. (a) Average latency reduction, (b) Throughput improvement.

5.7 Network Optimization

We isolate network effects by comparing DYNPIPE against FlexNN in edge-only deployment using ApacheBench [45] under identical resource constraints (Fig. 16). Unlike FlexNN’s memory-driven operator sharding, DYNPIPE jointly optimizes communication and computation-communication overlap through network-aware pipeline construction. Fig. 16a shows DYNPIPE reduces average inference time by 66.9% (LLaMA2-7B) and 52.8% (Whisper), while Fig. 16b demonstrates 3.6 $\times$ and 2.4 $\times$ throughput gains, confirming that strategic operator placement—not merely additional resources—drives performance improvement. EdgeShard is excluded because it inherently requires a cloud partition.

5.8 Overhead Analysis

Overhead in Bursty Workloads. Burst workloads expose edge-only deployment limitations through resource contention (Fig. 17). DYNPIPE’s hybrid architecture mitigates this via dynamic cloud offloading, limiting overhead to 3.1 $\times$ for LLaMA2-7B (62% reduction vs. FlexNN) and achieving near-zero overhead for Whisper. FlexNN exhibits 8.2 $\times$ and 20 $\times$ overhead respectively due to computational bottlenecks, while EdgeShard’s static partitioning yields 45 $\times$ overhead without adaptive load balancing.

Overhead in Different Phases. We analyze LLaMA2-7B overhead across workload phases (Fig. 18). DYNPIPE maintains stability through dynamic edge-cloud orchestration while FlexNN’s overhead escalates 2.7 $\times$ during accumulation phases

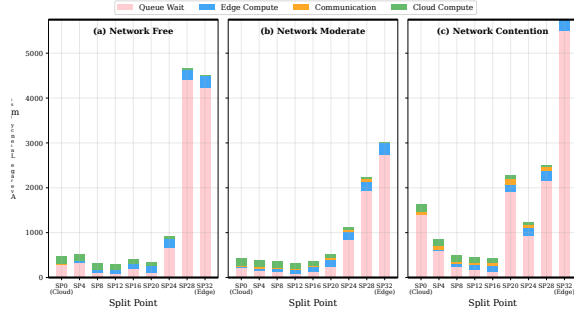


图 14: 三种网络状态下 QPS=5 下的分割点性能。最佳分割点从 SP=12 (网络自由) 转移到 SP=8 (网络竞争), 证明了动态边界自适应的必要性。

§4.1 根据经验: 在表 III 中的 9 个操作条件 (3 个负载级别 $\times$ 3 个网络状态) 中, 32 个候选层中仅出现 4 个不同的最佳分割点 ({4,8,12,16})。大小为 2 的投资组合 (例如 {4, 12}) 在争用情况下在 QPS=4 处错过了 SP=16 (最佳时间为 315 毫秒, 而 SP=12 为 428 毫秒, 损失为 36%), 在争用情况下在 QPS=5 处错过了 SP=8。规模为 4 的投资组合涵盖了所有观察到的最优值, 且剩余差距为零; 添加第五种配置在所有测试条件下都不会产生进一步的改进。这与理论预测一致: 对于均匀的 Transformer 块, 单调性崩溃足够强, 以至于 $|\bar{X}| = 4 \ll \bar{X} = 32$ 。具有异构层结构的模型可能需要更大的投资组合, 但增长仍然受到 $\bar{X}$ 而不是 $\bar{X}$ 的限制。

4) Heterogeneous Edge Impact

在边缘 GPU 吞吐量降低 30% 的实际硬件异构性下 (图 15), SP=12 实现了 215ms 的延迟, 而仅云 (SP=0) 的延迟为 179ms, 而仅边缘 (SP=32) 需要 255ms, 随着吞吐量损失在所有层上累积, 速度慢了 42%。然而, 在负载下, 边缘吞吐量的劣势主要由管道并行性的排队优势所主导。在 QPS=5 处, SP=12 的管道服务速率 ($\bar{X} \approx 8.9$ rps) 远远超过仅云 ($\bar{X} \approx 5.6$ rps) 和仅边缘 ($\bar{X} \approx 3.9$ rps), 与协作带来的 64% 吞吐量增益相比, 每层边缘开销成为次要因素。这验证了 DynoPipe 的计算平衡边界选择: LRP 算法在计算最佳边界时考虑了设备特定的吞吐量不对称性, 确保尽管硬件异构, 但管道瓶颈 ($\max(\bar{X}_{\text{edge}}, \bar{X}_{\text{cloud}})$) 最小化。

对启发式选择的敏感性。我们通过上面的多机制消融和投资组合分析来验证 LRP 参数的稳健性。少于四个候选者的投资组合会失去适应性 (例如, 缺少 SP=16 会导致 QPS=4 损失 36%; 参见表 III), 而超过四个候选者不会带来额外的好处, 与 $|\bar{X}| \leq \min(\bar{X}, \bar{X})$ 界限一致。滞后阈值 $\bar{X} \in [15\%, 20\%]$ 达到了良好的平衡: 降低它会导致微小波动的过度切换, 而提高它只会稍微延迟必要的变化。总体而言, 九种操作条件下的这些结果表明, 有限组合方法在现实的边缘云场景中是稳健的。

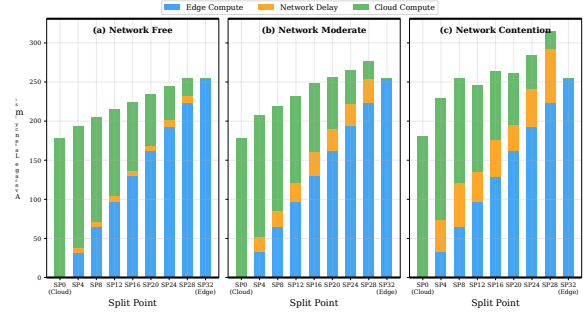


图 15: 异构边缘 GPU 的延迟细分 (速度降低 30%)。边缘计算损失随着分配给边缘的层数的增加而累积, 使得云密集的分割点对于单个请求更加高效, 而管道并行性有利于负载下的平衡分割。

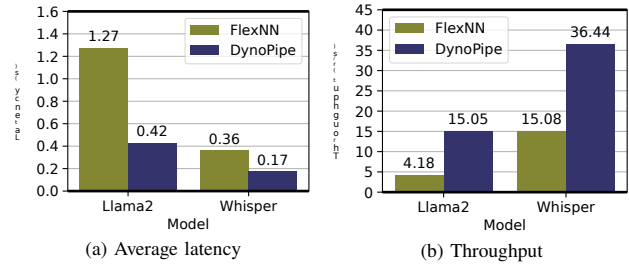


图 16: DynoPipe 和 FlexNN 在仅边缘部署中的性能比较。 (a) 平均延迟减少, (b) 吞吐量提高。

5.7 网络优化

我们通过在相同资源限制下使用 ApacheBench [45] 在仅边缘部署中比较 DynoPipe 与 FlexNN 来隔离网络效应 (图 16)。与 FlexNN 的内存驱动算子分片不同, DynoPipe 通过网络感知管道构建联合优化通信和计算通信重叠。图 16a 显示 DynoPipe 将平均推理时间减少了 66.9% (LLaMA2-7B) 和 52.8% (Whisper), 而图 16b 显示了 3.6 $\times$ 和 2.4 $\times$ 吞吐量增益, 证实了战略性算子放置 (而不仅仅是额外资源) 可以推动性能改进。EdgeShard 被排除在外, 因为它本质上需要云分区。

5.8 开销分析

突发工作负载的开销。突发工作负载通过资源争用暴露了仅限边缘的部署限制 (图 17)。DynoPipe 的混合架构通过动态云卸载缓解了这一问题, 将 LLaMA2-7B 的开销限制在 3.1 $\times$ (与 FlexNN 相比减少了 62%), 并且 Whisper 的开销几乎为零。由于计算瓶颈, FlexNN 分别表现出 8.2 $\times$ 和 20 $\times$ 开销, 而 EdgeShard 的静态分区在没有自适应负载平衡的情况下产生 45 $\times$ 开销。

不同阶段的开销。我们分析了 LLaMA2-7B 跨工作负载阶段的开销 (图 18)。DynoPipe 通过动态边缘云编排保持稳定, 而 FlexNN 的开销在累积阶段增加 2.7 $\times$

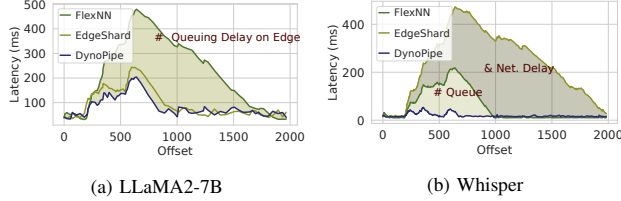


Fig. 17: Latency overhead comparison between DYNOPiPE and baseline systems. Overhead is measured as the difference from dedicated GPU execution time. (a) LLaMA2-7B. (b) Whisper.

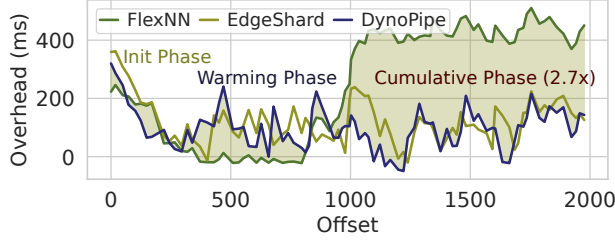


Fig. 18: Overhead of DYNOPiPE and baselines across different workload phases. The accumulation phase typically constitutes a significant portion of the system's lifecycle.

due to edge resource exhaustion. Portfolio generation completes offline in ~ 3 min per device pair (< 30 KB lookup tables). L1 hot boundary cache occupies ~ 150 MB GPU memory ($< 0.7\%$ of 24 GB); L2 host-RAM staging adds ~ 500 MB.

Worst-case migration: under simultaneous network contention (effective bandwidth 2.5 Gbps) and QPS=8 load, the P99 boundary migration latency is 85 ms, dominated by differential KV cache transfer (72 ms) plus parameter re-staging (13 ms). Under extreme conditions (bandwidth < 1 Gbps), the system activates the recomputation fallback (§4.3), bounding migration to 120 ms at $< 25\%$ additional compute cost.

Numerical Consistency. DYNOPiPE partitions models at transformer block boundaries, ensuring identical floating-point computations regardless of device placement. Cross-domain data consists solely of hidden-state activations transmitted without lossy compression. Model outputs are numerically identical across split-point configurations up to hardware floating-point non-determinism ($< 10^{-6}$ relative error). KV cache continuity during migration is preserved through differential synchronization and incremental reconstruction. LLaMA2-7B perplexity on WikiText-103 varies by < 0.02 ($< 0.3\%$) across SP=0/12/24/32 configurations.

6. Related Work

Edge-cloud Collaborative Serving. Edge-cloud collaborative inference systems distribute DL models across heterogeneous resources to balance latency, energy efficiency, and computational constraints [46–49]. Advanced frameworks employ reinforcement learning for adaptive model sharding and DAG optimization for computation-communication overlap [50–52]. Recent systems address privacy-latency trade-offs through secure computation protocols [53, 54] and LLM-specific static partitioning [27, 55]. Cheetah [30] protects input privacy by performing

inference directly over homomorphically encrypted data, but the cryptographic overhead results in latency several orders of magnitude beyond interactive serving requirements. However, these approaches fail under temporal resource volatility and migration overhead in heterogeneous edge-cloud environments. DYNOPiPE addresses these limitations through dynamic boundary adaptation that jointly optimizes placement, state management, and network contention.

Pipeline Parallelism. Existing pipeline parallelism frameworks [41, 56–62] focus on homogeneous data centers, while recent works [18, 26, 63–66] target workload and device heterogeneity but still assume stable network conditions. However, these approaches do not address the severe edge-cloud resource and network imbalance, which causes substantial pipeline idle time and makes static partitioning ineffective. DYNOPiPE addresses these challenges via dynamic, boundary-shifting pipelines that flexibly adapt computation placement to heterogeneous, volatile edge-cloud environments.

Model Inference Optimization. Edge inference has been improved through hardware-software co-design [67–69], intelligent caching [70–74], adaptive scheduling [75, 76], and model compression [11, 36, 77]. Throughput is further boosted by statistical multiplexing [7, 38, 39, 78–80] and long-context optimization [29]. Sarathi-Serve [7] optimizes throughput-latency trade-offs in homogeneous GPU clusters, while DYNOPiPE addresses the more challenging setting of cross-domain pipeline placement with heterogeneous edge-cloud resources. Yet, most approaches focus on isolated components and overlook system-level mismatches between edge constraints and LLM needs. In contrast, DYNOPiPE holistically coordinates computation placement, state management, and network utilization to bridge this gap.

7. Conclusion

We present DYNOPiPE, a collaborative inference system that addresses the fundamental challenge of deploying large models in resource-constrained edge environments through intelligent edge-cloud orchestration. By exploiting the heterogeneous communication patterns across model operators, DYNOPiPE constructs adaptive pipelines that strategically partition computation between edge devices and cloud resources.

Acknowledgments

We thank the anonymous reviewers and shepherd for their valuable feedback. This work is supported by the National Key R&D Program of China (No. 2025YFE0204100), Science and Technology Development Fund of Macao S.A.R (FDCT) under number 0074/2025/AMJ, Guangdong Basic and Applied Basic Research Foundation (No. 2023B1515130002). Xinyu Zhang and Baicheng Chen acknowledge the support of the Ericsson Endowed Chair Professorship at UC San Diego. This collaboration took place while Yanying Lin was a visiting PhD student at UC San Diego.

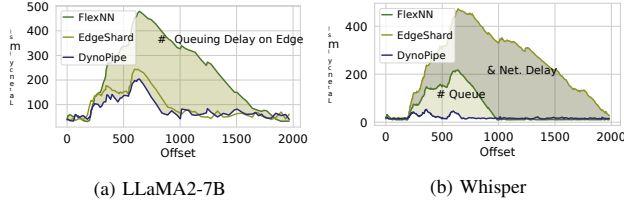


Fig. 17: Latency overhead comparison between DynoPipe and baseline systems. Overhead is measured as the difference from dedicated GPU execution time. (a) LLaMA2-7B. (b) 耳语。

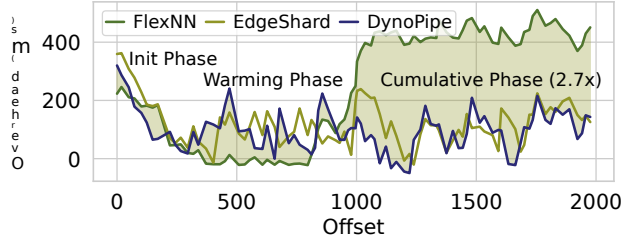


图 18: DynoPipe 的开销和不同工作负载阶段的基线。积累阶段通常构成系统生命周期的重要部分。

由于边缘资源耗尽。Portfolio generation completes offline in ~ 3 min per device pair (< 30 KB lookup tables). L1 hot boundary cache occupies ~ 150 MB GPU memory ($< 0.7\%$ of 24 GB); L2 主机 RAM 分段增加了 ~ 500 MB。

最坏情况迁移：在同时网络争用（有效带宽 2.5 Gbps）和 QPS=8 负载下，P99 边界迁移延迟为 85 ms，主要是差分 KV 缓存传输（72 ms）加上参数重新暂存（13 ms）。在极端条件下（带宽 < 1 Gbps），系统会激活重新计算回退（第 4.3 节），将迁移时间限制为 120 毫秒，而 $<$ 需要额外增加 25% 的计算成本。

数值一致性。DynoPipe 在变压器块边界处对模型进行分区，确保无论设备放置如何，都可以进行相同的浮点计算。跨域数据仅由未经有损压缩传输的隐藏状态激活组成。在硬件浮点非确定性（ $< 10^{-6}$ 相对误差）范围内，模型输出在数值上是相同的。通过差分同步和增量重建来保持迁移期间的 KV 缓存连续性。WikiText-103 上的 LLaMA2-7B 困惑度在 SP=0/12/24/32 配置中变化为 < 0.02 ($< 0.3\%$)。

六、相关工作

边云协同服务。边云协同推理系统跨异构资源分布深度学习模型，以平衡延迟、能源效率和计算约束[46-49]。先进的框架采用强化学习来实现自适应模型分片，并采用 DAG 优化来实现计算通信重叠 [50-52]。最近的系统通过安全计算协议 [53, 54] 和 LLM 特定的静态分区 [27, 55] 解决隐私延迟权衡问题。Cheetah [30] 通过执行以下操作来保护输入隐私

直接对同态加密数据进行推理，但加密开销导致延迟超出交互式服务要求几个数量级。然而，这些方法在异构边缘云环境中的时间资源波动和迁移开销下会失败。DynoPipe 通过动态边界适应来解决这些限制，共同优化布局、状态管理和网络争用。

管道并行性。现有的管道并行框架 [41, 56–62] 侧重于同构数据中心，而最近的工作 [18, 26, 63–66] 目标工作负载和设备异构性，但仍然假设稳定的网络条件。然而，这些方法并没有解决边缘云资源和网络严重不平衡的问题，这会导致大量管道空闲时间并使静态分区无效。DynoPipe 通过动态、边界转移管道来应对这些挑战，这些管道可以灵活地将计算放置适应异构、易变的边缘云环境。

模型推理优化。通过硬件-软件协同设计[67-69]、智能缓存[70-74]、自适应调度[75, 76]和模型压缩[11,36,77]改进了边缘推理。统计复用[7,38,39,78-80]和长上下文优化[29]进一步提高了吞吐量。Sarathi-Serve [7] 优化了同构 GPU 集群中的吞吐量-延迟权衡，而 DynoPipe 则解决了具有异构边缘云资源的跨域管道放置的更具挑战性的设置。然而，大多数方法都专注于孤立的组件，而忽略了边缘约束和法学硕士需求之间的系统级不匹配。相比之下，DynoPipe 整体协调计算布局、状态管理和网络利用率，以弥补这一差距。

七、结论

我们推出了 DynoPipe，这是一种协作推理系统，可通过智能边缘云编排解决在资源受限的边缘环境中部署大型模型的基本挑战。通过利用模型运算符之间的异构通信模式，DynoPipe 构建了自适应管道，在边缘设备和云资源之间战略性地划分计算。

致谢

我们感谢匿名审稿人和牧羊人的宝贵反馈。该工作得到国家重点研发计划（No. 2025YFE0204100）、澳门特别行政区科学技术发展基金（FDCT）（No. 0074/2025/AMJ）、广东省基础与应用基础研究基金（No. 2023B1515130002）的支持。张欣宇和陈百成感谢加州大学圣地亚哥分校爱立信讲席教授的支持。这次合作是在林彦英还是加州大学圣地亚哥分校的访问博士生期间进行的。

References

- [1] S. Han, S. Moon, T. Suh, J. Heo, and J.-Y. Kim, "Bless: Bandwidth and locality enhanced smem seeding acceleration for dna sequencing," in *51st ACM/IEEE Annu. Int. Symp. Comput. Archit. ISCA 2024 B. Aires Argent. June 29 - July 3 2024*. IEEE, 2024, pp. 582–596.
- [2] S. Rashidi, W. Won, S. Srinivasan, S. Sridharan, and T. Krishna, "Themis: A network bandwidth-aware collective scheduling policy for distributed training of dl models," in *ISCA 22 49th Annu. Int. Symp. Comput. Archit. N. Y. N. Y. USA June 18 - 22 2022*, V. Salapura, M. Zahran, F. Chong, and L. Tang, Eds. ACM, 2022, pp. 581–596.
- [3] X. Zhao, M. Jahre, Y. Tang, G. Zhang, and L. Eeckhout, "Nuba: Non-uniform bandwidth gpus," in *Proc. 28th ACM Int. Conf. Archit. Support Program. Lang. Oper. Syst. Vol. 2, ser. ASPLOS '23*. ACM, 2023-01-27, pp. 544–559.
- [4] F. Cangialosi, N. Agarwal, V. Arun, S. Narayana, A. Sarwate, and R. Netravali, "Privid: Practical, privacy-preserving video analytics queries," in *19th USENIX Symp. Networked Syst. Des. Implement. NSDI 2022 Renton WA USA April 4-6 2022*, ser. NSDI 2022, 2022, pp. 209–228.
- [5] M. L'ecuyer, R. Spahn, K. Vodrahalli, R. Geambasu, and D. Hsu, "Privacy accounting and quality control in the sage differentially private ml platform," in *Proc. 27th ACM Symp. Oper. Syst. Princ., ser. SOSP '19*. ACM, 2019-10-27, pp. 181–195.
- [6] X. Liu, L. Xie, Y. Wang, J. Zou, J. Xiong, Z. Ying, and A. V. Vasilakos, "Privacy and security issues in deep learning: A survey," vol. 9, pp. 4566–4593, 2021.
- [7] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, A. Tumanov, and R. Ramjee, "Taming throughput-latency tradeoff in llm inference with sarathi-serve," in *18th USENIX Symp. Oper. Syst. Des. Implement. OSDI 2024 St. Clara CA USA July 10-12 2024*, A. Gavrilovska and D. B. Terry, Eds. USENIX Association, 2024, pp. 117–134.
- [8] Y. Lin, Y. Li, S. Peng, Y. Tang, S. Luo, H. Shen, C. Xu, and K. Ye, "Quart: Latency-aware faas system for pipelining large model inference," in *2024 IEEE 44th Int. Conf. Distrib. Comput. Syst. ICDCS, 2024-07*, pp. 1–12.
- [9] R. Choudhary, J. Yu, C. Fletcher, and A. Morrison, "Speculative privacy tracking (spt): Leaking information from speculative execution without compromising privacy," in *MICRO-54 54th Annu. IEEEACM Int. Symp. Microarchitecture*, ser. MICRO '21. ACM, 2021-10-18, pp. 607–622.
- [10] T. Dong, Y. Meng, S. Li, G. Chen, Z. Liu, and H. Zhu, "Depth gives a false sense of privacy: {LLM} internal states inversion," in *34th USENIX Security Symposium (USENIX Security 25)*, 2025, pp. 1629–1648.
- [11] S. Kim, C. Hooper, A. Gholami, Z. Dong, X. Li, S. Shen, M. W. Mahoney, and K. Keutzer, "Squeezellm: Dense-and-sparse quantization," 2024-06-04.
- [12] H. Lu, L. Chang, C. Li, Z. Zhu, S. Lu, Y. Liu, and M. Zhang, "Distilling bit-level sparsity parallelism for general purpose deep learning acceleration," in *MICRO-54 54th Annu. IEEEACM Int. Symp. Microarchitecture*, ser. MICRO '21. ACM, 2021-10-18, pp. 963–976.
- [13] C. Xu, W. Zhou, T. Ge, F. Wei, and M. Zhou, "Bert-of-theseus: Compressing bert by progressive module replacing," in *Proc. 2020 Conf. Empir. Methods Nat. Lang. Process. EMNLP*. Association for Computational Linguistics, 2020-02-10.
- [14] C. Xu, K. Vora, and R. Gupta, "Pnp: Pruning and prediction for point-to-point iterative graph analytics," in *Proc. Twenty-Fourth Int. Conf. Archit. Support Program. Lang. Oper. Syst., ser. ASPLOS '19*. ACM, 2019-04-04, pp. 587–600.
- [15] A. M. Abdelmoniem, A. N. Sahu, M. Canini, and S. A. Fahmy, "Refl: Resource-efficient federated learning," in *Proc. Eighteenth Eur. Conf. Comput. Syst., ser. EuroSys '23*. ACM, 2023-05-08, pp. 215–232.
- [16] D. Chai, J. Zhang, L. Yang, Y. Jin, L. Wang, K. Chen, and Q. Yang, "Efficient decentralized federated singular vector decomposition," in *Proc. 2024 USENIX Annu. Tech. Conf. USENIX ATC 2024 St. Clara CA USA July 10-12 2024*, S. Bagchi and Y. Zhang, Eds. USENIX Association, 2024, pp. 1029–1047.
- [17] M. Xu, D. Cai, Y. Wu, X. Li, and S. Wang, "Fwdllm: Efficient federated finetuning of large language models with perturbed inferences," in *Proc. 2024 USENIX Annu. Tech. Conf. USENIX ATC 2024 St. Clara CA USA July 10-12 2024*, S. Bagchi and Y. Zhang, Eds. USENIX Association, 2024, pp. 579–596.
- [18] M. Adnan, Y. E. Maboud, D. Mahajan, and P. J. Nair, "Heterogeneous acceleration pipeline for recommendation system training," in *51st ACM/IEEE Annu. Int. Symp. Comput. Archit. ISCA 2024 B. Aires Argent. June 29 - July 3 2024*. IEEE, 2024, pp. 1063–1079.
- [19] S. Choi, S. Lee, Y. Kim, J. Park, Y. Kwon, and J. Huh, "Serving heterogeneous machine learning models on multi-gpu servers with spatio-temporal sharing," in *Proc. 2022 USENIX Annu. Tech. Conf. USENIX ATC 2022 Carlsbad CA USA July 11-13 2022*, ser. USENIX ATC 2022. USENIX Association, 2022, pp. 199–216.
- [20] C. Delimitrou and C. Kozyrakis, "Qos-aware scheduling in heterogeneous datacenters with paragon," vol. 31, no. 4, pp. 1–34, 2013-12.
- [21] S.-P. Yang, M. Kim, S. Nam, J. Park, J.-Y. Choi, E. H. Nam, E. Lee, S. Lee *et al.*, "Overcoming the memory wall with cxl-enabled ssds," in *Proc. 2023 USENIX Annu. Tech. Conf. USENIX ATC 2023 Boston MA USA July 10-12 2023*, ser. USENIX ATC 2023. USENIX Association, 2023, pp. 601–617.
- [22] A. Maricq, D. Duplyakin, I. Jimenez, C. Maltzahn, R. Stutsman, R. Ricci, and A. Klimovic, "Taming performance variability," in *13th USENIX Symp. Oper. Syst. Des. Implement. OSDI 2018 Carlsbad CA USA Oct. 8-10 2018*, ser. OSDI 2018, 2018, pp. 409–425.
- [23] R. Han, J. Wang, Q. Qi, H. Sun, C. Xu, Z. Wan, Z. Zhuang, Y. Yu *et al.*, "Netren: Service migration-driven network renaissance with synthesizing upyard configuration," in *Proc. 29th ACM Int. Conf. Archit. Support Program. Lang. Oper. Syst. Vol. 3, ser. ASPLOS '24*. ACM, 2024-04-27, pp. 708–721.
- [24] J. Jung, J. Kim, and J. Lee, "Deepum: Tensor migration and prefetching in unified memory," in *Proc. 28th ACM Int. Conf. Archit. Support Program. Lang. Oper. Syst. Vol. 2, ser. ASPLOS '23*. ACM, 2023-01-27, pp. 207–221.
- [25] M. Planeta, J. Bierbaum, L. S. D. Antony, T. Hoefler, and H. H.artig, "Migros: Transparent live-migration support for containerised rdma applications," in *Proc. 2021 USENIX Annu. Tech. Conf. USENIX ATC 2021 July 14-16 2021*, ser. USENIX ATC 2021. USENIX Association, 2021, pp. 47–63.
- [26] Z. Sun, H. Cao, Y. Wang, G. Feng, S. Chen, H. Wang, and W. Chen, "Adapipe: Optimizing pipeline parallelism with adaptive recomputation and partitioning," in *Proc. 29th ACM Int. Conf. Archit. Support Program. Lang. Oper. Syst. Vol. 3, ser. ASPLOS '24*. ACM, 2024-04-27, pp. 86–100.
- [27] M. Zhang, J. Cao, X. Shen, and Z. Cui, "Edgeshard: Efficient llm inference via collaborative edge computing," arXiv.org, 2024-05-23. [Online]. Available: <https://arxiv.org/abs/2405.14371v1>
- [28] J. Hu, W. Huang, W. Wang, Z. Li, T. Hu, Z. Liu, X. Chen, T. Xie *et al.*, "Efficient long-decoding inference with reasoning-aware attention sparsity," 2025-02-16.
- [29] Z. Yue, H. Zhuang, A. Bai, K. Hui, R. Jagerman, H. Zeng, Z. Qin, D. Wang *et al.*, "Inference scaling for long-context retrieval

References

- [1] S. Han, S. Moon, T. Suh, J. Heo 和 J.-Y. Kim, “Bless: 带宽和局部性增强了 DNA 测序的 smem 播种加速”, *51st ACM/IEEE Annu. Int. Symp. Comput. Archit. ISCA 2024 B. Aires Argent. June 29 - July 3 2024*, IEEE, 2024 年, 第 582–596 页。[2] S. Rashidi, W. Won, S. Srinivasan, S. Sridharan 和 T. Krishna, “Themis: 用于 dl 模型分布式训练的网络带宽感知集体调度策略”, 见 *ISCA 22 49th Annu. Int. Symp. Comput. Archit. N. Y. N. Y. USA June 18 - 22 2022*, V. Salapura, M. Zahran, F. Chong 和 L. Tang, 编辑。ACM, 2022 年, 第 581–596 页。[3] X. Zhao, M. Jahre, Y. Tang, G. Zhang 和 L. Eeckhout, “Nuba: 非均匀带宽 GPU”, 载于 *Proc. 28th ACM Int. Conf. Archit. Support Program. Lang. Oper. Syst. Vol. 2*, 序列号。ASPL OS '23. ACM, 2023 年 1 月 27 日, 第 544–559 页。[4] F. Cangialosi, N. Agarwal, V. Arun, S. Narayana, A. Sarwate 和 R. Netravali, “Privid: 实用、保护隐私的视频分析查询。”在 *19th USENIX Symp. Networked Syst. Des. Implement. NSDI 2022 Renton WA USA April 4-6 2022* 中, 爵士。NSDI 2022, 2022, 第 209–228 页。[5] M. L'ecuyer, R. Spahn, K. Vodrahali, R. Geambasu 和 D. Hsu, “圣人差异化私有机器学习平台中的隐私核算和质量控制”, 参见 *Proc. 27th ACM Symp. Oper. Syst. Princ., ser. SOSP '19*, ACM, 2019 年 10 月 27 日, 第 181–195 页。[6] X. Liu, L. Xie, Y. Wang, J. Zou, J. Xiong, Z. Ying, A. V. Vasilakos, “深度学习中的隐私和安全问题: 一项调查”, 卷。9, 第 4566–4593 页, 2021 年。[7] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, A. Tumanov 和 R. Ramjee, “用 sarathi-serve 驯服 llm 推理中的吞吐量-延迟权衡”, *18th USENIX Symp. Oper. Syst. Des. Implement. OSDI 2024 St. Clara CA USA July 10-12 2024*, A. 加夫里洛夫斯卡 (Gavrilovska) 和 D. B. 特里 (D. B. Terry), 编辑。USENIX 协会, 2024 年, 第 117–134 页。[8] Y. Lin, Y. Li, S. Peng, Y. Tang, S. Luo, H. Shen, C. Xu 和 K. Ye, “Quart: 用于流水线大型模型推理的延迟感知 faas 系统”, *2024 IEEE 44th Int. Conf. Distrib. Comput. Syst. ICDCS*, 2024 年 07 月, 第 1–12 页。[9] R. Choudhary, J. Yu, C. Fletcher 和 A. Morrison, “推测性隐私跟踪 (spt): 在不损害隐私的情况下从推测性执行中泄露信息”, *MICRO-54 54th Annu. IEEE/ACM Int. Symp. Microarchitecture*, 系列。微' 21. ACM M, 2021 年 10 月 18 日, 第 607–622 页。[10] T. Dong, Y. Meng, S. Li, G. Chen, Z. Liu 和 H. Zhu, “深度给人一种错误的隐私感: {LLM} 内部状态反转”, 载于 *34th USENIX Security Symposium (USENIX Security 25)*, 2025 年, 第 1629–1648 页。[11] S. Kim, C. Hooper, A. Gholami, Z. Dong, X. Li, S. Shen, M. W. Mahoney 和 K. Keutzer, “Squeezellm: 密集和稀疏量化”, 2024-06-04。[12] H. Lu, L. Chang, C. Li, Z. Zhu, S. Lu, Y. Liu 和 M. Zhang, “为通用深度学习加速提炼层级稀疏并行性”, 见 *MICRO-54 54th Annu. IEEE/ACM Int. Symp. Microarchitecture*, 系列。微' 21. ACM, 2021 年 10 月 18 日, 第 963–976 页。[13] C. Xu, W. Zhou, T. Ge, F. Wei 和 M. Zhou, “Bert-of-theseus: 通过渐进模块替换压缩 bert”, 见 *Proc. 2020 Conf. Empir. Methods Nat. Lang. Process. EMNLP*, 计算语言学协会, 2020-02-10。[14] C. Xu, K. Vora 和 R. Gupta, “Pnp: 点对点迭代图分析的修剪和预测”, *Proc. Twenty-Fourth Int. Conf. Archit. Support Program. Lang. Oper. Syst.*, 系列。ASPLOS '19. ACM, 2019-04-04, 第 587–600 页。[15] A. M. Abdelmoniem, A. N. Sahu, M. Canini 和 S. A. Fahmy, “Refl: 资源高效的联邦学习”, 载于 *Proc. Eighteenth Eur. Conf. Comput. Syst., ser. EuroSys '23*, ACM, 2023-05-08, 第 215–232 页。[16] D. Chai, J. 张, L. Yang, Y. Jin, L. Wang, K. Chen 和 Q. Yang, “高效去中心化联合奇异向量分解”, 载于 *Proc. 2024 USENIX Annu. Tech. Conf. USENIX ATC 2024 St. Clara CA USA July 10-12 2024*, S. Bagchi 和 Y. 张, 编辑。USENIX 协会, 2024 年, 第 1029–1047 页。[17] M. Xu, X. Li 和 S. Wang, “Fwdllm: 具有扰动推理的大语言模型的高效联合微调”, 参见 *Proc. 2024 USENIX Annu. Tech. Conf. USENIX ATC 2024 St. Clara CA USA July 10-12 2024*, S. Bagchi 和 Y. Zhang, 编辑。USENIX 协会, 2024 年, 第 579–596 页。[18] M. Adnan, Y. E. Maboud, D. Mahajan 和 P. J. Nair, “推荐系统训练的异构加速管道”, 参见 *51st ACM/IEEE Annu. Int. Symp. Comput. Archit. ISCA 2024 B. Aires Argent. June 29 - July 3 2024*, IEEE, 2024 年, 第 1063–1079 页。[19] S. Choi, S. Lee, Y. Kim, J. Park, Y. Kwon 和 J. Huh, “通过时空共享在多 GPU 服务器上提供异构机器学习模型。”在 *Proc. 2022 USENIX Annu. Tech. Conf. USENIX ATC 2022 Carlsbad CA USA July 11-13 2022* 中, 爵士。USENIX ATC 2022。USENIX 协会, 2022 年, 第 199–216 页。[20] C. Delimitrou 和 C. Kozyrakis, “Paragon 异构数据中心的 Qos 感知调度”, 第 1 卷。31, 没有。4, 第 1–34 页, 2013 年 12 月。[21] S.-P. 杨, M. Kim, S. Nam, J. Park, J.-Y. Choi, E. H. Nam, E. Lee, S. Lee *et al.*, “使用支持 cxl 的 SSD 克服内存墙。”在 *Proc. 2023 USENIX Annu. Tech. Conf. USENIX ATC 2023 Boston MA USA July 10-12 2023* 中, 爵士。USENIX ATC 2023。USENIX 协会, 2023 年, 第 601–617 页。[22] A. Maricq, D. Duplyakin, I. Jimenez, C. Maltzahn, R. Stutsman, R. Ricci 和 A. Klimovic, “驯服性能变异性”。在 *13th USENIX Symp. Oper. Syst. Des. Implement. OSDI 2018 Carlsbad CA USA Oct. 8-10 2018* 中, 爵士。OSDI 2018, 2018, 第 409–425 页。[23] R. Han, J. Wang, Q. Qi, H. Sun, C. Xu, Z. Wan, Z. Zhuang, Y. Yu *et al.*, “Netren: 服务迁移驱动的网络复兴与综合 upyard 配置”, 见 *Proc. 29th ACM Int. Conf. Archit. Support Program. Lang. Oper. Syst. Vol. 3*, 系列。ASPLOS '24. ACM, 2024 年 4 月 27 日, 第 708–721 页。[24] J. Jung, J. Kim 和 J. Lee, “Deepum: 统一内存中的张量迁移和预取”, *Proc. 28th ACM Int. Conf. Archit. Support Program. Lang. Oper. Syst. Vol. 2*, 系列。ASPLOS '23. ACM, 2023 年 1 月 27 日, 第 207–221 页。[25] M. Planeta, J. Bierbaum, L. S. D. Antony, T. Hoefler 和 H. H.artig, “Migras: 对容器化 rdma 应用程序的透明实时迁移支持。”在 *Proc. 2021 USENIX Annu. Tech. Conf. USENIX ATC 2021 July 14-16 2021* 中, 爵士。USENIX ATC 2021。USENIX 协会, 2021 年, 第 47–63 页。[26] Z. Sun, H. Cao, Y. Wang, G. Feng, S. Chen, H. Wang 和 W. Chen, “Adapipe: 通过自适应重新计算和分区优化管道并行性”, 见 *Proc. 29th ACM Int. Conf. Archit. Support Program. Lang. Oper. Syst. Vol. 3*, 系列。ASPLOS '24. ACM, 2024 年 4 月 27 日, 第 86–100 页。[27] M. 张, J. 曹, X. Shen 和 Z. Cui, “Edgeshard: 通过协作边缘计算进行高效 llm 推理”, arXiv.org, 2024-05-23。[在线]。可用: <https://arxiv.org/abs/2405.14371v1> [28] J. Hu, W. Huang, W. Wang, Z. Li, T. Hu, Z. Liu, X. Chen, T. Xie *et al.*, “具有推理感知注意力稀疏性的高效长解码推理”, 2025-02-16。[29] Z. Yue, H. Zhuang, A. Bai, K. Hui, R. Jagerman, H. Zeng, Z. qin, D. Wang *et al.*, “长上下文检索的推理缩放”

- augmented generation,” 2025-03-02.
- [30] B. Reagen, W.-S. Choi, Y. Ko, V. T. Lee, H.-H. S. Lee, G.-Y. Wei, and D. Brooks, “Cheetah: Optimizing and accelerating homomorphic encryption for private inference,” in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2021, pp. 26–39.
 - [31] T. Li, Y. Guo, and J. Fu, “Hybridcrypt-llm: Lightweight privacy for llm training and inference,” *Expert Systems with Applications*, p. 131632, 2026.
 - [32] Y. Jandali, R. Zhang, N. Sheybani, and F. Koushanfar, “Optimizing privacy-preserving primitives to support llm-scale applications,” in *2025 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. IEEE, 2025, pp. 1–9.
 - [33] D. Andreoletti, A. Rudi, E. Carpanzano, and T. Leidi, “Privacy-preserving llm inference in practice: A comparative survey of techniques, trade-offs, and deployability,” *Cryptology ePrint Archive*, 2026.
 - [34] Y. Gao, Z. Zeng, D. Du, S. Cao, P. Zhou, J. Qi, J. Lai, H. K.-H. So *et al.*, “Seerattention: Learning intrinsic sparse attention in your llms,” 2025-02-17.
 - [35] H. Jiang, Y. Li, C. Zhang, Q. Wu, X. Luo, S. Ahn, Z. Han, A. H. Abdi *et al.*, “Minference 1.0: Accelerating pre-filling for long-context llms via dynamic sparse attention,” 2024-10-30.
 - [36] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, “Smoothquant: Accurate and efficient post-training quantization for large language models,” 2024-03-29.
 - [37] Y. Zhang, I. n. Goiri, G. I. Chaudhry, R. Fonseca, S. Elnikety, C. Delimitrou, and R. Bianchini, “Faster and cheaper serverless computing on harvested resources,” in *Proc. ACM SIGOPS 28th Symp. Oper. Syst. Princ.*, ser. SOSP ’21. ACM, 2021-10-26, pp. 724–739.
 - [38] H. Zhang, Y. Tang, A. Khandelwal, and I. Stoica, “Shepherd: Serving dnns in the wild,” in *20th USENIX Symp. Networked Syst. Des. Implement. NSDI 2023 Boston MA April 17-19 2023*, ser. NSDI 2023. USENIX Association, 2023, pp. 787–808.
 - [39] X. Miao, C. Shi, J. Duan, X. Xi, D. Lin, B. Cui, and Z. Jia, “Spot-serve: Serving generative large language models on preemptible instances,” in *Proc. 29th ACM Int. Conf. Archit. Support Program. Lang. Oper. Syst. Vol. 2*, ser. ASPLOS ’24. ACM, 2024-04-27, pp. 1112–1127.
 - [40] B. Wu, R. Zhu, Z. Zhang, P. Sun, X. Liu, and X. Jin, “dlora: Dynamically orchestrating requests and adapters for lora llm serving,” in *18th USENIX Symp. Oper. Syst. Des. Implement. OSDI 2024 St. Clara CA USA July 10-12 2024*, A. Gavrilovska and D. B. Terry, Eds. USENIX Association, 2024, pp. 911–927.
 - [41] Z. Li, L. Zheng, Y. Zhong, V. Liu, Y. Sheng, X. Jin, Y. Huang, Z. Chen *et al.*, “Alpaserve: Statistical multiplexing with model parallelism for deep learning serving,” in *17th USENIX Symp. Oper. Syst. Des. Implement. OSDI 2023 Boston MA USA July 10-12 2023*, ser. OSDI 2023. USENIX Association, 2023, pp. 663–679.
 - [42] P. Patel, E. Choukse, C. Zhang, A. Shah, I. n. Goiri, S. Maleki, and R. Bianchini, “Splitwise: Efficient generative llm inference using phase splitting,” in *51st ACM/IEEE Annu. Int. Symp. Comput. Archit. ISCA 2024 B. Aires Argent. June 29 - July 3 2024*. IEEE, 2024, pp. 118–132.
 - [43] X. Li, Y. Li, Y. Li, T. Cao, and Y. Liu, “Flexnn: Efficient and adaptive dnn inference on memory-constrained edge devices,” in *Proc. 30th Annu. Int. Conf. Mob. Comput. Netw.*, ser. ACM MobiCom ’24. ACM, 2024-05-29, pp. 709–723.
 - [44] BentoML, “Key metrics for LLM inference,” LLM Inference Handbook, accessed: 2025-11-17. [Online]. Available: <https://bentoml.com/llm/inference-optimization/llm-inference-metrics>
 - [45] Apache, “Ab - apache http server benchmarking tool - apache http server version 2.4.” [Online]. Available: <https://httpd.apache.org/docs/2.4/programs/ab.html>
 - [46] Y. Kang, J. Hauswald, C. Gao, A. Rovinski, T. Mudge, J. Mars, and L. Tang, “Neurosurgeon: Collaborative intelligence between the cloud and mobile edge,” vol. 45, no. 1, pp. 615–629, 2017-05-11.
 - [47] H.-J. Jeong, H.-J. Lee, C. H. Shin, and S.-M. Moon, “Ionn: Incremental offloading of neural network computations from mobile devices to edge servers,” in *Proc. ACM Symp. Cloud Comput.*, ser. SoCC ’18. ACM, 2018-10-11, pp. 401–411.
 - [48] Y. Huang, F. Wang, F. Wang, and J. Liu, “Deepar: A hybrid device-edge-cloud execution framework for mobile deep learning applications,” in *IEEE INFOCOM 2019 - IEEE Conf. Comput. Commun. Workshop INFOCOM WKSHPS*, ser. INFOCOM 2019. IEEE, 2019-04, pp. 892–897.
 - [49] Z. Zhao, K. M. Barijough, and A. Gerstlauer, “Deepthings: Distributed adaptive deep learning inference on resource-constrained iot edge clusters,” vol. 37, no. 11, pp. 2348–2359, 2018-11.
 - [50] L. Wang, L. Xiang, J. Xu, J. Chen, X. Zhao, D. Yao, X. Wang, and B. Li, “Context-aware deep model compression for edge cloud computing,” in *2020 IEEE 40th Int. Conf. Distrib. Comput. Syst. ICDCS*, ser. ICDCS 2020. IEEE, 2020-11, pp. 787–797.
 - [51] S. Zhang, Y. Li, X. Liu, S. Guo, W. Wang, J. Wang, B. Ding, and D. Wu, “Towards real-time cooperative deep inference over the cloud and edge end devices,” vol. 4, no. 2, pp. 1–24, 2020-06-15.
 - [52] J. Ren, S. Rajbhandari, R. Y. Aminabadi, O. Ruwase, S. Yang, M. Zhang, D. Li, and Y. He, “Zero-offload: Democratizing billion-scale model training,” in *Proc. 2021 USENIX Annu. Tech. Conf. USENIX ATC 2021 July 14-16 2021*, ser. USENIX ATC 2021. USENIX Association, 2021, pp. 551–564.
 - [53] H. Birge-Lee, S. Yoo, B. Herber, J. Rexford, and M. Apostolaki, “Tango: Secure collaborative route control across the public internet,” in *21st USENIX Symp. Networked Syst. Des. Implement. NSDI 2024 St. Clara CA April 15-17 2024*, ser. NSDI 2024. USENIX Association, 2024.
 - [54] J. Liagouris, V. Kalavri, M. Faisal, and M. Varia, “Secrecy: Secure collaborative analytics in untrusted clouds,” in *20th USENIX Symp. Networked Syst. Des. Implement. NSDI 2023 Boston MA April 17-19 2023*, ser. NSDI 2023. USENIX Association, 2023, pp. 1031–1056.
 - [55] H. Jin and Y. Wu, “Ce-collm: Efficient and adaptive large language models through cloud-edge collaboration,” *arXiv preprint arXiv:2411.02829*, 2024.
 - [56] L. Zheng, Z. Li, H. Zhang, Y. Zhuang, Z. Chen, Y. Huang, Y. Wang, Y. Xu *et al.*, “Alpa: Automating inter- and intra-operator parallelism for distributed deep learning,” in *16th USENIX Symp. Oper. Syst. Des. Implement. OSDI 2022 Carlsbad CA USA July 11-13 2022*, ser. OSDI 2022, 2022, pp. 559–578.
 - [57] D. Narayanan, A. Harlap, A. Phanishayee, V. Seshadri, N. R. Devanur, G. R. Ganger, P. B. Gibbons, and M. Zaharia, “Pipedream: Generalized pipeline parallelism for dnn training,” in *Proc. 27th ACM Symp. Oper. Syst. Princ.*, ser. SOSP ’19. ACM, 2019-10-27, pp. 1–15.
 - [58] D. Narayanan, A. Phanishayee, K. Shi, X. Chen, and M. Zaharia, “Memory-efficient pipeline-parallel dnn training,” in *Proc. 38th Int. Conf. Mach. Learn. ICML 2021 18-24 July 2021 Virtual Event*, ser. ICML 2021. PMLR, 2021-07-01, pp. 7937–7947.
 - [59] Z. Bai, Z. Zhang, Y. Zhu, and X. Jin, “Pipeswitch: Fast pipelined context switching for deep learning applications,” in *14th USENIX Symp. Oper. Syst. Des. Implement. OSDI 2020 Virtual Event Novemb. 4-6 2020*, ser. OSDI 2020, 2020, pp. 499–514.
 - [60] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, “Outrageously large neural networks: The sparsely-gated mixture-of-experts layer,” 2017-01-23.

增强一代”，2025-03-02。[30] B. Reagen, W.-S. Choi, Y. Ko, V. T. Lee, H.-H. S. Lee, G.-Y. Wei 和 D. Brooks, “Cheetah: 优化和加速私有推理的同态加密”，见 *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*。IEEE, 2021 年, 第 177 页。26-39. T. Li, Y. Guo 和 J. Fu, “Hybrid crypt-llm: 用于 llm 训练和推理的轻量级隐私”，第 131632 页, 2026。[32] Y. Jandali, R. Zhang, N. Sheybani 和 F. Koushanfar, “优化隐私保护原语以支持 llm 规模”应用程序”，in *2025 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*。IEEE, 2025, 第 1-9 页。[33] D. Andreoletti, A. Rudi, E. Carpanzano 和 T. Leidi, “实践中的隐私保护 llm 推理: 技术、权衡和可部署性的比较调查”，*Cryptology ePrint Archive*, 2026 年。[34] Y. 高, Z. Zeng, D. Du, S. Cao, P. Zhou, J. Qi, J. Lai, H. K.-H. So *et al.*, “Seerattention: 在 llms 中学习内在稀疏注意力”，2025-02-17 [35] H. Jiang, Y. Li, C. 张, Q. Wu, X. Luo, S. Ahn, Z. Han, A. H. Abdi *et al.*, “Minference 1.0: 通过动态稀疏注意力加速长上下文 llms 的预填充”，2024-10-30。[36] G. Xiao, J. Lin, M. Sez Demouth, 和 S. Han, “Smoothquant: 准确高效的大语言模型训练后量化”，2024-03-29。[37] Y. Zhang, In.1.0 注意力加速长上下文 llms 的预填充。Goiri, G. I. Chaudhry, R. Fonseca, S. Elnikety, C. Delimitrou 和 R. Bianchini, “对收获的资源进行更快、更便宜的无服务器计算”，载于 *Proc. ACM SIGOPS 28th Symp. Oper. Syst. Princ.*, SOSOP '21, ACM, 2021 年 10 月 26 日, 第 724-739 页 [38] H. 张, Y. Tang, A. Khandelwal 和 I. Stoica, “牧羊人: 在野外服务 dnns”。在 *20th USENIX Symp. Networked Syst. Des. Implement. NSDI 2023 Boston MA April 17-19 2023* 中, 爵士。NSDI 2023。USENIX 协会, 2023 年, 第 787-808 页。[39] X. Miao, C. Shi, J. Duan, X. Xi, 和 Z. Jia, “Spot-serve: 在可抢占实例上提供生成大语言模型”，*Proc. 29th ACM Int. Conf. Archit. Support Program. Lang. Oper. Syst. Vol. 2*, 系列。ASPLOS '24。ACM, 2024 年 4 月 27 日, 第 1112-1127 页。[40] B. Wu, R. Zhu, Z. Zhang, P. Sun, X. Liu 和 X. Jin, “dlora: 动态编排用于 lora llm 服务的请求和适配器”，载于 *18th USENIX Symp. Oper. Syst. Des. Implement. OSDI 2024 St. Clara CA USA July 10-12 2024*, A. Gavrilovska 和 D. B. Terry, 编辑。USENIX 协会, 2024 年, 第 911-927 页。[41] Z. Li, L. Cheng, Y. zhong, V. Liu, Y. Sheng, X. Jin, Y. Huang, Z. Chen *et al.*, “Alpaserve: 用于深度学习服务的模型并行统计复用。”在 *17th USENIX Symp. Oper. Syst. Des. Implement. OSDI 2023 Boston MA USA July 10-12 2023* 中, 爵士。OSDI 2023。USENIX 协会, 2023 年, 第 63-679 页。[42] P. Patel, E. Choukse, C. 张, A. Shah, I. n. Goiri, S. Maleki 和 R. Bianchini, “Splitwise: 使用相分裂的高效生成 llm 推理”，*51st ACM/IEEE Annu. Int. Symp. Comput. Archit. ISCA 2024 B. Aires Argent. June 29 - July 3 2024*。IEEE, 2024 年, 第 118-132 页。[43] X. Li, Y. Li, Y. Li, T. Cao 和 Y. Liu, “Flexnn: 内存受限边缘设备上的高效自适应 DNN 推理”，*Proc. 30th Annu. Int. Conf. Mob. Comput. Netw.*, 系列。ACM MobiCom '24。ACM, 2024 年 5 月 29 日, 第 709-723 页。[44] BentoML, “LLM 推理的关键指标”，LLM 推理手册, 访问时间: 2025 年 11 月 17 日。[在线]。可用: <https://bentoml.com/llm/inference-optimization/llm-inference-metrics> [45] Apache, “Ab - apache http 服务器基准测试工具

- apache http 服务器版本 2.4。”[在线]。可用: <https://httpd.apache.org/docs/2.4/programs/ab.html> [46] Y. Kang, J. Hauswald, C. Gau, A. Rovinski, T. Mudge, J. Mars 和 L. Tang, “神经外科医生: 云与移动边缘之间的协作智能”，第 1 卷。45, 没有。1, 第 615-629 页, 2017-05-11。[47] H.-J. 郑, H.-J. Lee, C. H. Shin 和 S.-M. Moon, “Ionn: 将神经网络计算从移动设备增量卸载到边缘服务器”，*Proc. ACM Symp. Cloud Comput.*, 系列。SoCC '18。ACM, 2018 年 10 月 11 日, 第 401-411 页。[48] Y. Huang, F. Wang, F. Wang 和 J. Liu, “Deepar: 用于移动深度学习应用程序的混合设备-边缘-云执行框架”，*IEEE INFOCOM 2019 - IEEE Conf. Comput. Commun. Workshop INFOCOM WKSHPs*, 系列。INFOCOM 2019。IEEE, 2019-04, 第 892-897 页。[49] Z. Zhao, K. M. Barijough 和 A. Gerstlauer, “Deepthings: 资源受限的物联网边缘集群上的分布式自适应深度学习推理”，卷。37, 没有。11, 第 2348-2359 页, 2018 年 11 月。[50] L. Wang, L. Xiang, J. Xu, J. Chen, X. Zhao, D. Yao, X. Wang 和 B. Li, “边缘云计算的上下文感知深度模型压缩”，见 *2020 IEEE 40th Int. Conf. Distrib. Comput. Syst. ICDCS*, 系列。ICDCS 2020。IEEE, 2020-11, 第 787-797 页。[51] 张 S., 李 Y., 刘 X., 郭 S., W. Wang, J. Wang, B. Ding, 和 D. Wu, “面向云端和边缘设备的实时协作深度推理”，第 1 卷。4, 没有。2, 第 1-24 页, 2020 年 6 月 15 日。[52] J. Ren, S. Rajbhandari, R. Y. Aminabadi, O. Ruwase, S. Yang, M. 张, D. Li 和 Y. He, “零卸载: 十亿规模模型训练民主化”。在 *Proc. 2021 USENIX Annu. Tech. Conf. USENIX ATC 2021 July 14-16 2021* 中, 爵士。USENIX ATC 2021。USENIX 协会, 2021 年, 第 551-564 页。[53] H. Birge-Lee, S. Yoo, B. Herber, J. Rexford 和 M. Apostolaki, “Tango: 跨公共互联网的安全协作路由控制”。在 *21st USENIX Symp. Networked Syst. Des. Implement. NSDI 2024 St. Clara CA April 15-17 2024* 中, 爵士。NSDI 2024。USENIX 协会, 2024。[54] J. Liagouris, V. Kalavri, M. Faisal 和 M. Varia, “保密: 不受信任的云中的安全协作分析。”在 *20th USENIX Symp. Networked Syst. Des. Implement. NSDI 2023 Boston MA April 17-19 2023* 中, 爵士。NSDI 2023。USENIX 协会, 2023 年, 第 1031-1056 页。[55] H. Jin 和 Y. Wu, “Ce-collm: 通过云边缘协作实现高效和自适应大语言模型”，*arXiv preprint arXiv:2411.02829*, 2024。[56] L. Cheng, Z. Li, H. Zhuang, Y. Z. Huang, Z. Chen, Y. Huang, Y. Wang, Y. Xu *et al.*, “Alpa: 自动化分布式深度算子间和算子内并行性学习。”在 *16th USENIX Symp. Oper. Syst. Des. Implement. OSDI 2022 Carlsbad CA USA July 11-13 2022* 中, 爵士。OSDI 2022, 2022, 第 559-578 页。[57] D. Narayanan, A. Harlap, A. Phanishayee, V. Seshadri, N. R. Devanur, G. R. Ganger, P. B. Gibbons 和 M. Zaharia, “Pipedream: dn 训练的广义管道并行性”，参见 *Proc. 27th ACM Symp. Oper. Syst. Princ.*, ser. SOSOP '19。ACM, 2019 年 10 月 27 日, 第 1-15 页。[58] D. Narayanan, A. Phanishayee, K. Shi, X. Chen 和 M. Zaharia, “内存高效的管道并行 dnn 训练”。在 *Proc. 38th Int. Conf. Mach. Learn. ICML 2021 18-24 July 2021 Virtual Event* 中, 爵士。ICML 2021。PMLR, 2021-07-01, 第 7937-7947 页。[59] Z. Bai, Z. Zhang, Y. Zhu 和 X. Jin, “Pipeswitch: 深度学习应用的快速流水线上下文切换。”在 *14th USENIX Symp. Oper. Syst. Des. Implement. OSDI 2020 Virtual Event Novemb. 4-6 2020* 中, 爵士。OSDI 2020, 2020, 第 499-514 页。[60] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton 和 J. Dean, “极其庞大的神经网络: 稀疏门控混合专家层”，2017 年 1 月 23 日。

- [61] R. Y. Aminabadi, S. Rajbhandari, A. A. Awan, C. Li, D. Li, E. Zheng, O. Ruwase, S. Smith *et al.*, “Deepspeed- inference: Enabling efficient inference of transformer models at unprecedented scale,” in *SC22 Int. Conf. High Perform. Comput. Netw. Storage Anal.*, ser. SC 2022. IEEE, 2022-11, pp. 46:1–46:15.
- [62] S. Fan, Y. Rong, C. Meng, Z. Cao, S. Wang, Z. Zheng, C. Wu, G. Long *et al.*, “Dapple: A pipelined data parallel approach for training large models,” in *Proc. 26th ACM SIGPLAN Symp. Princ. Pract. Parallel Program.*, ser. PPoPP ’21. ACM, 2021-02-17, pp. 431–445.
- [63] I. Rocha, N. Morris, L. Y. Chen, P. Felber, R. Birke, and V. Schiavoni, “Pipetune: Pipeline parallelism of hyper and system parameters tuning for deep learning clusters,” in *Proc. 21st Int. Middlew. Conf.*, ser. Middleware ’20. ACM, 2020-12-07, pp. 89–104.
- [64] T. Chen, A. Kubicek, L. Huang, and T. Hoefler, “Crosspipe: Towards optimal pipeline schedules for cross-datacenter training,” in *Proc. 2025 USENIX Annu. Tech. Conf. USENIX ATC 2025 Boston MA USA July 7-9 2025*, D. Altinb’uken and R. Stutsman, Eds. USENIX Association, 2025, pp. 1089–1108.
- [65] Z. Sun, S. Chen, Y. Wang, J. Sha, G. Feng, and W. Chen, “Mepipe: Democratizing llm training with memory-efficient slice-level pipeline scheduling on cost-effective accelerators,” in *Proc. Twent. Eur. Conf. Comput. Syst. EuroSys 2025 Rotterdam Neth. 30 March 2025 - 3 April 2025*. ACM, 2025, pp. 1263–1278.
- [66] Y. Tan, C. Tan, Z. Mi, and H. Chen, “Pipellm: Fast and confidential large language model services with speculative pipelined encryption,” in *Proc. 30th ACM Int. Conf. Archit. Support Program. Lang. Oper. Syst. Vol. 1 ASPLOS 2025 Rotterdam Neth. 30 March 2025 - 3 April 2025*, L. Eeckhout, G. Smaragdakis, K. Liang, A. Sampson, M. A. Kim, and C. J. Rossbach, Eds. ACM, 2025, pp. 843–857.
- [67] M. Liu, S. Luo, K. Han, B. Yuan, R. F. DeMara, and Y. Bai, “An efficient real-time object detection framework on resource-constricted hardware devices via software and hardware co-design,” in *2021 IEEE 32nd Int. Conf. Appl.-Specif. Syst. Archit. Process. ASAP*, ser. ASAP 2021. IEEE, 2021-07, pp. 77–84.
- [68] Y. Yu, T. Zhao, K. Wang, and L. He, “Light-opu: An fpga-based overlay processor for lightweight convolutional neural networks,” in *Proc. 2020 ACM SIGDA Int. Symp. Field-Program. Gate Arrays*, ser. FPGA ’20. ACM, 2020-02-23, pp. 122–132.
- [69] L. Guo, W. Choe, and F. X. Lin, “Sti: Turbocharge nlp inference at the edge via elastic pipelining,” in *Proc. 28th ACM Int. Conf. Archit. Support Program. Lang. Oper. Syst. Vol. 2*, ser. ASPLOS ’23. ACM, 2023-01-27, pp. 791–803.
- [70] H. Wu, F. Lyu, C. Zhou, J. Chen, L. Wang, and X. Shen, “Optimal uav caching and trajectory in aerial-assisted vehicular networks: A learning-based approach,” vol. 38, no. 12, pp. 2783–2797, 2020-12.
- [71] T. Guo, R. J. Walls, and S. S. Ogden, “Edgeserve: Efficient deep learning model caching at the edge,” in *Proc. 4th ACM/IEEE Symp. Edge Comput.*, ser. SEC ’19. ACM, 2019-11-07, pp. 313–315.
- [72] U. Drolia, K. Guo, J. Tan, R. Gandhi, and P. Narasimhan, “Cachier: Edge-caching for recognition applications,” in *2017 IEEE 37th Int. Conf. Distrib. Comput. Syst. ICDCS*, ser. ICDCS 2017. IEEE, 2017-06, pp. 276–286.
- [73] Y. Liu, H. Li, Y. Cheng, S. Ray, Y. Huang, Q. Zhang, K. Du, J. Yao *et al.*, “Cachegen: Fast context loading for language model applications via kv cache streaming,” 2024-08-04.
- [74] F. Strati, S. Mcallister, A. Phanishayee, J. Tarnawski, and A. Klimovic, “Dejavu: Kv-cache streaming for fast, fault-tolerant generative llm serving,” 2024-03-04.
- [75] S. S. Shubha and H. Shen, “Adainf: Data drift adaptive scheduling for accurate and slo-guaranteed multiple-model inference serving at edge servers,” in *Proc. ACM SIGCOMM 2023 Conf.*, ser. ACM SIGCOMM ’23. ACM, 2023-09-10, pp. 473–485.
- [76] R. Bhardwaj, Z. Xia, G. Ananthanarayanan, J. Jiang, Y. Shu, N. Karianakis, K. Hsieh, P. Bahl *et al.*, “Ekya: Continuous learning of video analytics models on edge compute servers,” in *19th USENIX Symp. Networked Syst. Des. Implement. NSDI 2022 Renton WA USA April 4-6 2022*, ser. NSDI 2022. USENIX Association, 2022, pp. 119–135.
- [77] Y. Lin, H. Tang, S. Yang, Z. Zhang, G. Xiao, C. Gan, and S. Han, “Qserve: W4a8kv4 quantization and system co-design for efficient llm serving,” 2024-05-10.
- [78] S. Ahmad, H. Guan, B. D. Friedman, T. Williams, R. K. Sitaraman, and T. Woo, “Proteus: A high-throughput inference-serving system with accuracy scaling,” in *Proc. 29th ACM Int. Conf. Archit. Support Program. Lang. Oper. Syst. Vol. 1*, ser. ASPLOS ’24. ACM, 2024-04-27, pp. 318–334.
- [79] A. Gujarati, R. Karimi, S. Alzayat, W. Hao, A. Kaufmann, Y. Vigfusson, and J. Mace, “Serving dnns like clockwork: Performance predictability from the bottom up,” in *14th USENIX Symp. Oper. Syst. Des. Implement. OSDI 2020 Virtual Event Novemb. 4-6 2020*, ser. OSDI 2020. USENIX Association, 2020, pp. 443–462.
- [80] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, “Orca: A distributed serving system for transformer-based generative models,” in *16th USENIX Symp. Oper. Syst. Des. Implement. OSDI 2022 Carlsbad CA USA July 11-13 2022*, ser. OSDI 2022. USENIX Association, 2022, pp. 521–538.

- [61] R. Y. Aminabadi, S. Rajbhandari, A. A. Awan, C. Li, D. Li, E. Cheng, O. Ruwase, S. Smith *et al.*, “Deepspeed-inference: 以前所未有的规模实现变压器模型的高效推理”, in *SC22 Int. Conf. High Perform. Comput. Netw. Storage Anal.*, ser. SC 2022. IEEE, 2022 年 11 月, 第 46:1–46:15. [62] S. Fan, Y. Rong, C. Meng, Z. Cao, S. Wang, Z. Zheng, C. Wu, G. Long *et al.*, “Dapple: 一种用于训练大型模型的管道数据并行方法”, 见 *Proc. 26th ACM SIGPLAN Symp. Princ. Pract. Parallel Program.*, ser. PPoPP’ 21. ACM, 2021 年 2 月 17 日, 第 431–445 页. [63] I. Rocha, N. Morris, L. Y. Chen, P. Felber, R. Birke 和 V. Schiavoni, “Pipetune: 深度学习集群的超级和系统参数调整的管道并行性”, 见 *Proc. 21st Int. Middlew. Conf.*, ser. 中间件’ 20. ACM, 2020 年 12 月 7 日, 第 89–104 页. [64] T. Chen, A. Kubicek, L. Huang 和 T. Hoefler, “Crosspipe: 迈向跨数据中心训练的最佳管道调度”, 见 *Proc. 2025 USENIX Annu. Tech. Conf. USENIX ATC 2025 Boston MA USA July 7-9 2025*, D. Altinb“uken 和 R. Stutsman, Eds. USENIX Association, 2025 年, 第 1089–1108 页. [65] Z. Sun, S. Chen, Y. Wang, J. Sha, G. Feng 和 W. Chen, “Mepipe: 在经济高效的加速器上利用内存高效的切片级管道调度实现 llm 训练的大众化”, 载于 *Proc. Twent. Eur. Conf. Comput. Syst. EuroSys 2025 Rotterdam Neth. 30 March 2025 - 3 April 2025*, ACM, 2025 年, 第 1263–1278 页. [66] Y. Tan, C. H. Chen, “PipelLM: 具有推测性的快速且保密的大语言模型服务”。管道加密”, 载于 *Proc. 30th ACM Int. Conf. Archit. Support Program. Lang. Oper. Syst. Vol. 1 ASPLOS 2025 Rotterdam Neth. 30 March 2025 - 3 April 2025*, L. Eeckhout, G. Smaragdakis, K. Liang, A. Sampson, M. A. Kim 和 C. J. Rossbach, Eds. ACM, 2025, 第 843–857 页. [67] M. Liu, S. Luo, K. Han, B. Yuan, R. F. DeMara 和 Y. Bai, “通过软件和硬件协同设计在资源有限的硬件设备上实现高效的实时对象检测框架”, 载于 *2021 IEEE 32nd Int. Conf. Appl.-Specif. Syst. Archit. Process. ASAP*, ASAP 2021. IEEE, 2021-07, 第 77-84 页. [68] Y. Yu, T. Zhu, K. Wang 和 L. He, “Light-opu: 用于轻量级卷积神经网络的基于 fpga 的覆盖处理器”, 载于 *Proc. 2020 ACM SIGDA Int. Symp. Field-Program. Gate Arrays, FPGA’ 20*, 2020 年 2 月 23 日, 第 122–132 页. L. Guo, W. Choe 和 F. X. Lin, “Sti: 通过弹性流水线增强边缘 nlp 推理”, 载于 *Proc. 28th ACM Int. Conf. Archit. Support Program. Lang. Oper. Syst. Vol. 2, ASPLOS’ 23*. 2023-01-27, 第 791-803 页. [70] H. Wu, F. Lyu, C. Zhou, J. Chen, L. Wang 和 X. Shen, “空中辅助车辆网络中的最优无人机缓存和轨迹: 基于学习的方法”, 第 38 卷, 第 12 期, 第 2783-2797 页, 2020 年 12 月. [71] T. Guo, R. J. Walls 和 S. S. Ogden, “Edgeserve: 边缘的高效深度学习模型缓存”, 载于 *Proc. 4th ACM IEEE Symp. Edge Comput., SEC’ 19*, 2019 年 11 月 7 日, 第 313-315 页. Narasimhan, “Cachier: 识别应用程序的边缘缓存”, 2017 IEEE 37th Int. Conf. Distrib. Comput. Syst. ICDCS, ser. ICDCS 2017. IEEE, 2017-06, 第 276–286 页. [73] Y. Liu, H. Li, Y. Cheng, S. Ray, Y. Huang, Q. Zhu, K. Du, J. Yao *et al.*, “Cachegen: 语言模型应用程序的快速上下文加载” 通过 kv 缓存流”, 2024-08-04. [74] F. Strati, S. Mcallister, A. Phanishayee, J. Tarnawski 和 A. Klimov, “Dejavu: 用于快速、容错生成 llm 服务的 Kv 缓存流”, 2024-03-04. [75] S. S. Shubha 和 H. Shen, “Adainf: 数据漂移自适应调度用于在边缘服务器上提供准确且有保证的多模型推理”, 见 *Proc. ACM SIGCOMM 2023 Conf.*, ser. ACM SIGCOMM ’23. ACM, 2023-09-10, 第 473–485 页. [76] R. Bhardwaj, Z. Xia, G. Ananthanarayanan, J. Jiang, Y. Shu, N. Karianakis, K. Hsieh, P. Bahl *et al.*, “Ekya: 边缘计算服务器上视频分析模型的持续学习。” 在 *19th USENIX Symp. Networked Syst. Des. Implement. NSDI 2022 Renton WA USA April 4-6 2022* 中, 爵士. NSDI 2022. USENIX 协会, 2022 年, 第 119–135 页. [77] Y. Lin, H. Tang, S. Yang, Z. Zhu, G. Xiao, C. Gan 和 S. Han, “Qserve: W4a8kv4 量化和系统协同设计, 实现高效的 llm 服务”, 2024-05-10. [78] S. Ahmad, H. Guan, B. D. Friedman, T. Williams, R. K. Sitaraman 和 T. Woo, “Proteus: 具有精度缩放功能的高吞吐量推理服务系统”, 见 *Proc. 29th ACM Int. Conf. Archit. Support Program. Lang. Oper. Syst. Vol. 1, ser. A SPLOS’ 24*. ACM, 2024 年 4 月 27 日, 第 318–334 页. [79] A. Gujarati, R. Karimi, S. Alzayat, W. hao, A. Kaufmann, Y. Vigfusson 和 J. Mace, “像发条一样服务 dns: 自下而上的性能可预测性。” 在 *14th USENIX Symp. Oper. Syst. Des. Implement. OSDI 2020 Virtual Event Novemb. 4-6 2020* 中, 爵士. OSDI 2020. USENIX 协会, 2020 年, 第 443–462 页. [80] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim 和 B.-G. Chun, “Orca: 基于变压器的生成模型的分布式服务系统。” 在 *16th USENIX Symp. Oper. Syst. Des. Implement. OSDI 2022 Carlsbad CA USA July 11-13 2022* 中, 爵士. OSDI 2022. USENIX 协会, 2022 年, 第 521–538 页.